

Feedback-Guided Memory Adaptation for Correcting Persistent Persona Errors in AI Companions



THE UNIVERSITY OF
SYDNEY

Author: PEIXUAN ZHONG

Supervisor: CHANG XU

Associate Supervisor: Dr. Lijun Chang

A thesis submitted in fulfilment of the requirements for the degree of Master of
Philosophy

School of Computer Science
Faculty of Engineering
The University of Sydney
Australia

Submission Date: 29 June 2026

Statement or originality

I hereby declare that, to the best of my knowledge, this thesis is the result of my own work. It has not been submitted for any degree or any other academic purposes. I certify that the intellectual content of this thesis is my own, and any assistance received in its preparation, as well as all referenced sources, have been appropriately acknowledged.

SIGNATURE: PEIXUAN ZHONG

Authorship Attribution Statement

This thesis does not contain any material that has been published or accepted for publication by myself, either alone or in collaboration with others. All work presented in this thesis is original and has not been previously submitted for publication.

In addition to the authorship attribution statements above, in cases where I am not the corresponding author of a published item, permission to include the published material has been granted by the corresponding author.

Student Name: Peixuan Zhong, Signature: Peixuan Zhong, Date: 2026/06/28

As supervisor for the candidature upon which this thesis is based, I can confirm that the authorship attribution statements above are correct.

Supervisor Name: Chang Xu, Signature: _____, Date: 2026/06/28

Declaration

This thesis reports a controlled study of Feedback-Guided Memory Adaptation for persona correction in AI-companion prototypes. The experimental results are derived from the accompanying reproducibility artifacts. The method is an external prompt-memory mechanism; it does not modify base-model parameters, replace model-level alignment, or constitute a real-user deployment study.

Abstract

LLM-based AI companions are expected to carry a persona across repeated interactions, so persistent memory errors are harder to ignore than ordinary single-turn mistakes. A representative failure occurs when the system receives an incorrect persona prior and continues to repeat it after feedback. This thesis studies that wrong-prior recovery problem in controlled AI-companion settings. It proposes Feedback-Guided Memory Adaptation (FGMA), a system-level prompt-memory mechanism that turns structured diagnostic feedback into external memory entries. Each entry records the error type, failed-response summary, corrective feedback, corrected fact or rule, negative constraint, migration scope, and retrieval keywords. Retrieved entries are inserted into later prompts as explicit correction context. FGMA does not update base-model parameters.

The evaluation compares five feedback conditions: no memory, positive-only memory, error-diagnostic memory, mixed memory, and a Direct Correction Baseline. The experiments use synthetic or anonymized personas, controlled wrong-prior scenarios, Qwen-family models, cross-architecture models, multi-seed robustness checks, and rubric-based LLM-assisted judging. Error-diagnostic memory improves correction success and reduces repeated persona errors compared with positive-only memory. Mixed memory is close to error-diagnostic memory in the main run and slightly stronger in some cross-architecture settings, while the Direct Correction Baseline remains a practical comparison condition.

The claims are deliberately bounded. The work does not provide deployed user-engagement evidence, real-person avatar consent evidence, or human preference evaluation. Its contribution is narrower: structured diagnostic feedback can serve as a useful external memory signal for correcting persistent persona errors in reproducible AI-companion prototypes.

Acknowledgements

The author acknowledges the guidance, review, and technical support received during the research process.

Contents

1	Introduction	1
1.1	Problem Context	1
1.2	From Customized AI Companion System to Memory Adaptation Research	2
1.3	Motivation	2
1.4	Research Gap	3
1.5	Research Questions and Hypotheses	3
1.6	Contributions	4
1.7	Thesis Structure	4
2	Related Work	5
2.1	AI Companions and Persona-Based Dialogue	5
2.2	Persona Consistency Versus Factual Correctness	6
2.3	Long-Term Memory in Conversational Agents	6
2.4	Memory Failure Modes	7
2.5	Human Feedback and Preference Learning	7
2.6	Feedback Granularity	8
2.7	Error Feedback and Self-Correction	8
2.8	Prompt-Memory Adaptation	9
2.9	Evaluation of AI Companion Systems	9
2.10	Memory Governance and Correction Ownership	9
2.11	Positioning FGMA Against Adjacent Mechanisms	10
2.12	Research Gap Restated	10
2.13	Gap for This Study	11
3	System Design	12
3.1	System Overview	12
3.2	Relationship to the Original AI Companion Project	13
3.3	Persona Profile Construction	13

3.4	Response Generation Pipeline	13
3.5	External Memory Store	14
3.6	Feedback Collection and Memory Update	14
3.7	TUI Research Prototype	14
3.8	Logging and Reproducibility	15
3.9	Summary	15
4	Feedback-Guided Memory Adaptation	16
4.1	Motivation	16
4.2	Notation	16
4.3	Pipeline	17
4.4	Feedback Conditions	18
4.5	Memory Schema	19
4.6	Error Taxonomy	20
4.7	Retrieval and Scoping	21
4.8	Prompt Reconstruction	22
4.9	Algorithm	23
4.10	Why C4 Is a Baseline Rather Than a Ceiling	23
4.11	Design Rationale	23
4.12	Implementation Boundary	24
4.13	Summary	24
5	Experimental Setup	25
5.1	Study Design	25
5.2	Dataset Scale	25
5.3	Feedback Conditions	26
5.4	Models	26
5.5	Prompt Templates	27
5.6	Metrics	27
5.7	Statistical Analysis	28
5.8	Prompt Leakage and Token Budget Audits	28
5.9	TUI Demonstration Protocol	29
5.10	Reproducibility	29
6	Results and Analysis	30
6.1	Overall Results	30
6.2	Error-Diagnostic Memory Versus Positive-Only Memory	32

6.3	Mixed Memory	32
6.4	Direct Correction Baseline	33
6.5	Cross-Architecture Results	33
6.5.1	Qwen-Family Behaviour	34
6.5.2	Gemma and Llama Behaviour	34
6.5.3	Mistral Behaviour	35
6.6	Multi-Seed Robustness	35
6.7	Prompt Ablation	36
6.8	Memory Component Ablation	37
6.9	LLM-Assisted Judge Results	38
6.10	Qualitative Case Studies	38
6.11	Token Budget and Token-Matched Controls	39
6.12	Relationship Between Results and Hypotheses	40
6.13	Summary	41
7	Discussion	42
7.1	Why Error-Diagnostic Feedback Helps	42
7.2	Positive Feedback Is Not Useless	42
7.3	Implications for AI Companion Design	43
7.4	Relationship to Preference Learning	43
7.5	Limitations	43
7.6	Ethics	44
7.7	Threats to Validity	44
7.7.1	Internal Validity	44
7.7.2	Construct Validity	45
7.7.3	External Validity	45
7.7.4	Statistical Conclusion Validity	45
7.8	Practical Deployment	45
7.8.1	Conflict Resolution	45
7.8.2	Memory Decay and Staleness	46
7.8.3	Cost Controls	46
7.8.4	User-Facing Controls	46
7.9	Scope of Evidence	46
7.10	Directions for Future Work	46
7.11	Summary	47
8	Conclusion	48

A	Prompt Templates	53
A.1	C0 Prompt Template	53
A.2	C1 Prompt Template	53
A.3	C2 Prompt Template	54
A.4	C3 Prompt Template	54
A.5	C4 Prompt Template	54
B	Persona and Scenario Examples	55
B.1	Case Study: C2 Home-City Wrong-Prior Recovery	55
B.2	Case Study: Memory Omission	55
B.3	Case Study: Contradiction	56
C	TUI Screenshot Sequence	57
D	Detailed Tables and Reproducibility	60
D.1	Project Repository and Artifact Link	60
D.2	Artifact Overview	60
D.3	Raw Logs and Thesis-Facing Tables	60
D.4	Reproducibility Commands	61
D.5	Model and Label Notes	61
D.6	Reference Verification Register	61

List of Figures

3.1	System architecture for the controlled FGMA research prototype.	12
3.2	TUI memory-entry view.	14
4.1	End-to-end FGMA wrong-prior recovery pipeline.	18
4.2	Internal structure of Feedback-Guided Memory Adaptation.	20
6.1	Main wrong-prior recovery results across feedback conditions.	31
6.2	Cross-architecture wrong-prior recovery by condition.	33
6.3	Prompt strategy ablation for wrong-prior recovery.	36
6.4	Memory component ablation.	37
6.5	Token-matched non-corrective memory control.	40
C.1	Persona profile loading screen.	57
C.2	Dialogue screen showing a wrong-prior response.	57
C.3	Structured feedback input screen.	58
C.4	Prompt reconstruction screen.	58
C.5	Follow-up correction success screen.	58
C.6	Evaluation dashboard screen.	59

List of Tables

4.1	Feedback conditions used in the controlled wrong-prior recovery experiments. . . .	19
4.2	Core fields in an FGMA error-diagnostic memory entry.	20
4.3	Error taxonomy used by the FGMA feedback interface.	21
5.1	Relationship between raw dialogue logs and evaluated follow-up turns.	26
5.2	Simplified model summary for the main body. Exact runtime details are provided in the reproducibility appendix.	27
6.1	Main Qwen-family results by condition.	31
6.2	Planned paired binary statistical comparisons.	32
6.3	Condition-level cross-architecture validation results.	34
6.4	Multi-seed Qwen-family robustness results.	35
6.5	Rubric-based LLM-assisted judge summary by dataset. Factual, memory-use, and coherence columns are 1–5 rubric scores where higher is better; judge success and repeat error are rates. Scores are automatic judge outputs, not real-user scores. . . .	38
6.6	Mapping between hypotheses and empirical evidence.	41

Abbreviations

AI	Artificial intelligence
FGMA	Feedback-Guided Memory Adaptation
LLM	Large language model
TUI	Terminal user interface
RAG	Retrieval-augmented generation
RLHF	Reinforcement learning from human feedback

Chapter 1

Introduction

1.1 Problem Context

AI companions differ from single-turn assistants because their value depends on continuity. A search assistant can answer a question and then disappear from the user’s mental model. A companion is expected to preserve a recognizable persona, remember stable facts, maintain a suitable interaction style, and recover gracefully when corrected. This expectation makes memory failures more visible. A wrong answer in a normal question-answering setting is often a local failure. A wrong persona prior in a companion can become a repeated identity failure, because the system may reuse the incorrect value whenever the relevant persona attribute is queried.

Persona continuity has been studied in dialogue systems for many years, from early social interaction systems to persona-grounded neural conversation models [1], [6], [7]. Recent LLM-based systems make the issue more consequential because persona facts, retrieved memories, dialogue history, and system instructions are combined at inference time. These sources can conflict. A profile may contain a stale city, a memory may preserve an obsolete preference, or a previous answer may create a misleading conversational precedent. Once the wrong value enters the prompt context, the model can treat it as authoritative.

The failure studied here is wrong-prior repetition. A companion may be supplied with an incorrect profile fact, such as a home city, preferred beverage, signature project, or style rule. The first answer may repeat the wrong value. The more serious problem appears later, when a follow-up turn repeats the same value after feedback has already been given. At that point the user is not just seeing a factual mistake; they are seeing a correction fail to persist.

Wrong-prior repetition is narrow enough to evaluate without losing the companion-specific nature of the problem. The correct value and the corrupted value can be specified in advance. Follow-up questions can be paraphrased or delayed. Responses can be scored for both correction success and recurrence of the wrong value. That makes the task more measurable than broad companion

satisfaction, but less artificial than a one-off factual QA check.

The failure also exposes a tension between prompt authority and feedback authority. A profile field in the prompt may appear authoritative to the model, even when a later correction is more reliable. The memory system has to represent that correction strongly enough to compete with the corrupted prior. FGMA does this by storing diagnostic memory in a separate prompt block instead of expecting the model to infer the correction from a positive example.

1.2 From Customized AI Companion System to Memory Adaptation Research

The original project motivation was a customized AI-companion system. That broader application context includes persona profiles, style control, safety boundaries, memory storage, and user-specific continuity. For the thesis, the work is narrowed to one research question inside that system space: how a companion can recover when its persona memory or prompt context contains an incorrect prior. This narrower framing replaces unsupported backbone-comparison claims with a reproducible evaluation of a concrete memory adaptation mechanism.

The application setting and the research prototype are connected, but the prototype is deliberately smaller. It does not try to deliver a production companion platform. It isolates one mechanism that such a platform would still need: correction memory that can distinguish a successful response from a previously repeated error. The controlled prototype makes it possible to test the same persona, scenario, model, and follow-up structure under different feedback conditions.

That boundary matters methodologically and ethically. A production companion would need consent management, deletion controls, safety moderation, and long-term user studies. The research prototype uses synthetic or anonymized personas and controlled scenarios, so the memory mechanism can be studied without implying real deployment or real-person avatar consent.

The narrowing also makes the contribution easier to defend. A broad customized-companion project can become an implementation report if every subsystem is treated as equally central. By focusing on memory correction, the thesis can define a precise failure mode, state hypotheses, compare conditions, and interpret statistical evidence. The original companion context remains, but it motivates a focused research question rather than a claim of product completeness.

1.3 Motivation

Positive feedback is common in interactive systems. A user can like a response, save an example, or rate an answer as helpful. Positive examples may be useful for style preservation and for reinforcing

desirable interaction patterns. However, positive feedback alone is ambiguous when the system must recover from an incorrect prior. If a later answer is good, the system may not know whether it was good because the fact was corrected, because the tone improved, or because the wrong fact happened not to appear.

Error-diagnostic feedback carries more information. It can say what failed, what the correct value is, what value should be avoided, how broadly the correction should migrate, and which retrieval cues should activate the memory. The question is whether that extra structure measurably helps wrong-prior recovery. The issue is not whether feedback in general is useful; it is whether feedback that records an error and a negative constraint is more effective than memory that stores only positive examples.

1.4 Research Gap

Prior work covers many of the ingredients. Persona-based dialogue shows that explicit persona descriptions can make dialogue more consistent [6], [7]. Retrieval-augmented generation shows how external information can enter the prompt [18]–[22]. Long-term memory systems store and retrieve prior interaction content [23]–[25]. Human-feedback research studies preference signals at scale [26]–[31], and self-correction work studies revision after an error [33]–[35].

What is less well isolated is the comparison that matters here: positive-only memory versus error-diagnostic memory when the persona prior is wrong. In a companion setting, answering correctly once is not enough. The incorrect prior has to stop resurfacing. The experiments keep the wrong-prior setting fixed and vary the feedback memory condition.

1.5 Research Questions and Hypotheses

The work is organized around three questions. First, does structured error-diagnostic memory improve correction success compared with positive-only memory? Second, does error-diagnostic memory reduce repeated persona errors across follow-up turns? Third, are the observed trends directionally stable across model families, seeds, and rubric-based judge checks?

The corresponding hypotheses are conservative. H1 is that error-diagnostic memory will improve correction success because it stores the corrected value and the avoided value. H2 is that error-diagnostic memory will reduce repeated error because it gives prompt reconstruction an explicit negative constraint. H3 is that mixed memory will perform at least comparably to error-only memory, but any additional gain will depend on whether positive examples complement the correction rather than distract from it.

1.6 Contributions

The contribution is deliberately focused. The thesis formalizes wrong-prior recovery as a controlled AI-companion memory problem and proposes FGMA, a prompt-memory adaptation mechanism that stores diagnostic feedback as structured external memory rather than updating model weights. Around that method, it implements a reproducible research prototype, including a terminal inspection interface for qualitative analysis, and reports controlled experiments across feedback conditions, model families, seeds, and LLM-assisted judge checks.

The work is not presented as a new model architecture, a replacement for RLHF, or evidence from deployed user engagement. Its claim is narrower and testable: in the studied wrong-prior recovery setting, diagnostic memory is a more useful adaptation signal than positive-only memory.

There is also a practical negative result. Positive-only memory is plausible as a lightweight feedback mechanism, but it fails to recover wrong priors in the controlled setting. That matters because companion systems often treat saved good examples or liked responses as personalization memory. The experiments show that this representation is too weak when the system must suppress a known incorrect value.

1.7 Thesis Structure

Chapter 2 reviews related work on persona dialogue, long-term memory, feedback learning, self-correction, prompt-memory adaptation, and evaluation. Chapter 3 describes the system design and the relationship between the original companion context and the controlled research prototype. Chapter 4 presents the FGMA method. Chapter 5 defines the experimental setup. Chapter 6 reports results and analysis. Chapter 7 discusses implications, limitations, ethics, and threats to validity. Chapter 8 concludes the thesis.

Chapter 2

Related Work

2.1 AI Companions and Persona-Based Dialogue

AI companions build on a long history of conversational systems that invite users to treat software as socially meaningful. ELIZA demonstrated that even simple pattern-based interaction could produce a sense of dialogue [1], and later work on media equation effects argued that people often respond socially to computational systems [2]. Modern LLM-based companions are technically very different, but the continuity expectation remains: users expect the system to preserve identity-like facts and interaction style across time.

Persona-based dialogue research made this expectation explicit. PersonaChat introduced profiles as grounding information for dialogue agents [6], while persona-based neural conversation models explored how persona descriptions can influence generated responses [7]. These systems showed that persona conditioning can make dialogue more coherent, but they also exposed a persistent challenge: a model may rely too strongly on the supplied persona text, even when that text is incomplete or wrong.

Open-domain chatbot research broadened the setting from persona QA to general conversation [9]–[12]. These systems combine conversational fluency, retrieval, safety filters, and sometimes memory. For AI companions, persona consistency is not only a benchmark property. It is part of the user-facing relationship. If a companion repeatedly claims the wrong home city or contradicts a stable preference, the failure damages perceived continuity.

Persona QA and long-term companion continuity are related but not the same problem. Persona QA asks whether a model can answer a question from a profile. Companion continuity asks whether the system can maintain, revise, and govern persona information across repeated interaction. Wrong-prior recovery belongs to the second problem. The model must not merely read a profile; it must recover when the profile or memory context contains an incorrect prior.

2.2 Persona Consistency Versus Factual Correctness

Persona consistency is broader than factual correctness. A factual metric can ask whether the system says "Lisbon" rather than "Barcelona." Persona consistency also asks whether the system preserves stable identity facts, respects style traits, maintains boundaries, and avoids contradictions across turns. In an AI companion, these dimensions interact. A response can be factually correct but stylistically inconsistent, or warm in tone but factually wrong.

This is why the primary metrics focus on correction success and repeated error. Wrong-prior recovery needs a measurable target. At the same time, the surrounding analysis treats persona continuity as a system property. A correction mechanism should not simply overwrite text; it should help the companion maintain a stable persona representation while remaining editable and accountable.

The same distinction explains why a wrong persona prior is more damaging than an ordinary factual error. If a question-answering system gives one wrong fact, the user may treat the answer as a retrieval failure. If a companion repeatedly claims the wrong persona fact, the user may interpret the system as having an unstable identity or memory. The recovery mechanism needs to prevent recurrence, not merely produce a corrected answer once.

2.3 Long-Term Memory in Conversational Agents

Long-term memory systems attempt to extend agent behavior beyond the immediate context window. Generative agents used memory streams, reflection, and retrieval to simulate believable behavior over time [23]. MemGPT proposed operating-system-like memory management for LLM agents [24]. MemoryBank studied long-term memory for LLM-based agents [25]. These systems motivate the idea that external memory can support continuity without retraining the base model.

However, memory storage is not memory governance. A memory system can store useful facts, but it can also store stale, contradictory, over-generalized, or unsafe information. Retrieval failure can hide a relevant correction. Conflict failure can retrieve a correction and a wrong prior at the same time. Scope failure can apply a correction too broadly. Staleness failure can preserve a fact after it has changed. These are not merely implementation details; they determine whether a companion's memory is trustworthy.

FGMA sits inside this memory-governance problem. It does not propose that memory alone solves continuity. Instead, it asks what kind of memory entry is needed after the system has already made a persona error. A correction memory should contain more than a positive example. It should record the failed response, the corrected fact or rule, the value to avoid, the scope of migration, and retrieval cues.

2.4 Memory Failure Modes

Long-term memory can fail in several ways. Stale memory preserves information that was once true but no longer applies. Conflict failure occurs when a profile and a memory entry disagree. Retrieval failure occurs when the relevant memory exists but is not selected for the current prompt. Over-generalization applies a correction beyond its intended scope, while under-scoping prevents a relevant correction from being used. Memory omission occurs when a salient fact is not recalled. A particularly subtle failure occurs when the system stores an incorrect response as a positive example, thereby reinforcing the wrong prior.

These failure modes are the reason for the FGMA fields. The corrected fact handles stale or incorrect content. The negative constraint handles conflict with the wrong prior. Migration scope handles over-generalization and under-scoping. Retrieval keywords help with retrieval failure and paraphrased follow-up. The failed-response summary makes it possible to audit whether a stored memory was created for the right reason. In that sense, FGMA is a memory-governance schema for correction, not just another memory buffer.

The failure-mode framing also clarifies why the experiments compare positive-only memory against diagnostic memory. Positive memory can store a fluent example, but it may not state that an earlier value is wrong. If the wrong value remains in the profile or dialogue context, the positive example may be too weak to resolve the conflict. Diagnostic memory is designed to make the conflict explicit.

2.5 Human Feedback and Preference Learning

Human-feedback methods have improved model behavior by learning from preferences or demonstrations [26]–[31]. RLHF and related methods operate at a global model-behavior level: they collect preference information and use it to adjust model behavior or selection policies. These methods are highly relevant to alignment, instruction following, and general response quality.

The FGMA setting is smaller and more local. A wrong-prior correction may concern one user’s profile or one synthetic persona in a controlled evaluation. Updating model weights for every small correction would be impractical in many deployed systems and undesirable when the correction should remain scoped. Local memory adaptation is better understood as complementary to preference learning rather than as a replacement for it.

This distinction also affects the semantics of feedback. Preference feedback often says which answer is better. Diagnostic correction says what fact or rule was wrong and what should not be repeated. In wrong-prior recovery, the negative part of the signal is important. A system that only stores a good answer may still lack an explicit representation of the avoided prior.

2.6 Feedback Granularity

Feedback varies in granularity. Binary feedback indicates approval or disapproval. Preference feedback compares alternatives. Positive examples preserve desirable behavior. Direct correction supplies a corrected value. Diagnostic error feedback goes further by describing the failure, the corrected value, the avoided value, and the intended scope. A structured memory update then turns that diagnostic feedback into a reusable artifact.

The comparison in this thesis is about feedback structure, not just feedback presence. C1 and C2 both involve memory, but they store different information. C1 stores positive examples, while C2 stores diagnostic correction entries. The large gap between these conditions suggests that the form of feedback matters. For wrong-prior recovery, the useful signal is not simply "this was good" or "this was bad," but "this value was wrong, this value is correct, and this correction applies here."

Granularity also affects governance. A direct correction may be enough for a single factual question, but it may not record why the correction was made or how broadly it should apply. A structured diagnostic entry can support auditing and later conflict resolution. This makes feedback granularity a central design variable for AI companion memory.

2.7 Error Feedback and Self-Correction

Self-correction and verbal feedback methods show that LLMs can improve outputs when given critique or reflective context. Self-Refine uses iterative self-feedback to revise responses [33]. Reflexion stores verbal feedback for language agents [34]. ReAct combines reasoning and acting traces to improve task behavior [35]. These works support the broader idea that natural-language feedback can influence future behavior.

FGMA differs in its recurrence target. Revising one answer is not the same as preventing a persona error from recurring later. A companion may produce a corrected answer immediately after feedback, but then drift back to the wrong prior when asked a paraphrased or delayed follow-up. FGMA treats correction as reusable memory, not merely as local revision. The structured memory entry is designed to be retrieved when the same persona fact or related cue appears again.

The method also separates positive and error-diagnostic feedback. Without that separation, all feedback becomes undifferentiated text, and success is hard to attribute: it could come from positive examples, explicit correction, negative constraints, or simply longer prompts. The experimental conditions isolate these mechanisms.

2.8 Prompt-Memory Adaptation

Prompting, retrieval-augmented prompting, prompt tuning, and prefix tuning all show that model behavior can be changed without ordinary full-model retraining [18], [46], [47], [50]. Retrieval-augmented generation inserts external evidence into the context [18]–[22]. Prompt tuning and prefix tuning learn continuous prompt parameters, while FGMA uses transparent natural-language memory entries that can be inspected and edited.

Prompt-memory adaptation is powerful but brittle. Instructions can conflict, earlier context can dominate later corrections, and retrieved passages may be ignored. This brittleness motivates structured prompt reconstruction. FGMA separates persona, dialogue, positive memory, error memory, and safety constraints into distinct prompt blocks. The separation does not guarantee perfect compliance, but it makes the adaptation mechanism auditable.

The argument does not depend on a new claim about Transformer internals. The relevant mechanism is system-level: external memory plus prompt reconstruction. That makes the approach portable across open-weight model families in principle, while also making its limitations visible.

2.9 Evaluation of AI Companion Systems

Evaluation of AI companions is difficult because continuity, trust, style, and safety are partly experiential. Factual metrics can measure whether a corrected fact appears. Repeated-error metrics can measure whether the wrong value recurs. Persona consistency and coherence metrics can approximate style stability. LLM-as-judge methods can provide rubric-based assessment, but they are not substitutes for human-user studies [39]–[42].

Automatic binary metrics are the primary evidence because wrong-prior recovery is measurable in controlled scenarios. The thesis also reports heuristic persona, tone, and coherence metrics, plus LLM-assisted judge checks. These are treated as approximations, not as real-user assessment or engagement evidence.

2.10 Memory Governance and Correction Ownership

The literature on memory-enabled agents often emphasizes capability: the system can remember, retrieve, summarize, or reflect. For AI companions, governance is equally important. A memory that cannot be corrected may become a source of repeated harm. A memory that generalizes too broadly may apply one user’s correction to another user’s context. A memory that cannot be inspected may leave users unable to understand why a companion changed its answer.

Correction ownership is a design problem in its own right. A user correction should not become a global truth unless the user intends that scope. A persona-specific correction should remain attached to the relevant persona. A temporary correction should be allowed to expire. Existing memory systems motivate the storage and retrieval infrastructure; this thesis focuses on the narrower question of how a corrective memory entry should be structured once an error has been identified.

This governance view also clarifies the role of negative constraints. A corrected value alone may say what to answer, but it does not record what earlier value caused the failure. In a companion, the system may see both values again: the wrong value in an old dialogue turn and the corrected value in a later memory. A negative constraint gives the prompt constructor a way to resolve that conflict.

2.11 Positioning FGMA Against Adjacent Mechanisms

FGMA is adjacent to several mechanisms but not identical to them. It is not retrieval-augmented generation in the narrow sense of retrieving external factual documents. Its memory entries are produced by feedback about the system’s own prior response. It is not prompt tuning or prefix tuning because it does not learn continuous parameters. It is not preference learning because it does not optimize a global policy. It is not self-correction in the single-answer sense because the target is recurrence prevention across follow-up turns.

The closest conceptual neighbors are verbal feedback and agent memory. FGMA adopts the idea that feedback can be represented in language, but it adds a schema that distinguishes error type, corrected fact, avoided value, scope, and retrieval cues. This schema makes the feedback more reusable than an unstructured note. It also makes the memory easier to audit because a reader can see what claim the memory is supposed to enforce.

2.12 Research Gap Restated

The reviewed work shows that persona grounding, external memory, feedback learning, and self-correction are active research areas. The open question here is narrower: whether structured diagnostic error feedback is more useful than positive-only memory for a repeated persona correction task. The thesis answers that question empirically in a controlled setting, with a claim kept narrow enough for the available evidence.

2.13 Gap for This Study

The literature gives useful pieces: persona grounding, memory, feedback learning, self-correction, retrieval, and evaluation. What it does not settle is whether a correction should be stored as a positive example or as a diagnostic error memory when the underlying persona prior is wrong. FGMA is built around that smaller question.

Chapter 3

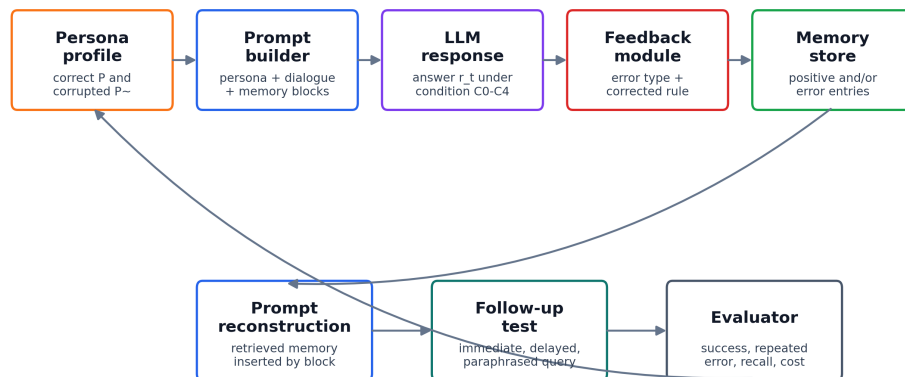
System Design

3.1 System Overview

The system design separates the original AI-companion motivation from the controlled research prototype. At the application level, an AI companion requires persona profiles, style constraints, safety boundaries, memory, and dialogue generation. At the research level, the thesis isolates a narrower pipeline: inject a wrong prior, generate a response, collect structured feedback, update external memory, reconstruct the prompt, and test follow-up recovery.

Figure 3.1 summarizes the system architecture. The key design choice is transparency: persona profiles, memory entries, prompt blocks, and logs remain inspectable artifacts. That is why the prototype is suitable for research analysis even though it is not a production companion system.

FGMA research prototype: transparent prompt-memory adaptation without changing base-model parameters



All turns are logged for reproducibility; the TUI exposes persona loading, feedback entry, memory update, prompt reconstruction, and dashboard views.

Figure 3.1: System architecture for the controlled FGMA research prototype.

3.2 Relationship to the Original AI Companion Project

The original project was concerned with customized AI companions. That broader system would include user-facing personalization, avatar identity, safety policies, and persistent interaction memory. The thesis narrows the contribution to a correction mechanism within that space. This is a methodological refinement rather than a retreat from the original motivation: a customized companion is not credible if it cannot correct persistent persona errors.

The research prototype evaluates a subsystem that would sit inside a larger companion architecture. It does not claim to solve all companion design problems. It studies whether structured diagnostic feedback is a better memory signal than positive-only feedback when the persona context contains an incorrect prior.

3.3 Persona Profile Construction

Each controlled run begins with a persona profile containing stable facts, style traits, and safety boundaries. A wrong prior is injected into a target fact field. The correct profile is denoted by P , while the corrupted prompt-facing profile is denoted by $\tilde{P}$. This notation makes the failure explicit: the model receives a profile that contains a wrong value, while the evaluator knows the correct value.

Persona profiles are synthetic or anonymized. This avoids claims about real-person consent and keeps the evaluation focused on memory adaptation. The profile schema includes factual fields such as home city or preference, style fields such as tone, and safety fields such as disclosure boundaries. The wrong-prior test changes the factual field while preserving the rest of the persona.

3.4 Response Generation Pipeline

The response generation pipeline assembles the prompt from persona information, recent dialogue, retrieved memory, scenario metadata, and the user query. In the first turn, conditions without relevant memory tend to repeat the corrupted prior because it is present in $\tilde{P}$. After feedback, conditions with diagnostic memory can retrieve a correction and insert it into the prompt.

The model response is logged with the prompt condition, model identifier, persona identifier, scenario identifier, and evaluator outputs. These logs support repeated analysis without rerunning the model. The thesis does not modify raw logs during polishing.

3.5 External Memory Store

The memory store is external to the model. It contains entries produced from feedback and successful examples, depending on the condition. External memory has two advantages for this thesis. First, it makes the adaptation mechanism transparent. Second, it allows per-persona corrections without changing the base model.

The memory store also creates risks. If a correction is over-scoped, it may affect the wrong persona. If the wrong prior is stored as a positive memory, it may reinforce the error. The FGMA schema is designed to reduce these risks by recording migration scope and negative constraints.

3.6 Feedback Collection and Memory Update

Feedback is collected as structured information rather than a single free-form rating. The feedback panel records the error type, corrected fact or rule, migration scope, and optional explanatory text. In C2 and C3, this feedback becomes an error-diagnostic memory entry. In C1 and C3, successful responses can also be stored as positive memory.

This structure is useful for reproducibility. A later reader can inspect what the system believed was wrong, what correction was stored, and what retrieval keywords were used. It also supports qualitative analysis because memory entries can be compared with later prompt reconstructions.

3.7 TUI Research Prototype

The terminal user interface is a research prototype for inspection and demonstration. It is not the main experimental engine. Its purpose is to show how persona loading, dialogue, feedback, memory update, prompt reconstruction, follow-up testing, and dashboard summary fit together. Figure 3.2 shows the memory-entry view, which is the most relevant TUI screen for the method.



Figure 3.2: TUI memory-entry view showing the structured correction fields used by FGMA.

Most TUI screenshots are placed in Appendix C. This keeps the main chapter focused on design while preserving the screenshots needed for thesis demonstration and reproducibility.

3.8 Logging and Reproducibility

Every experimental turn is logged with condition, persona, scenario, model, prompt context, response, feedback state, and metrics. The logs support aggregate tables, significance checks, and case studies. Reproducibility artifacts include scripts for table generation, figures, TUI replay, and statistical checks. Detailed commands and file paths are kept in the appendix rather than in the main argument.

3.9 Summary

The design chapter narrows the companion-system setting down to the interfaces needed for the experiment: persona construction, response generation, structured feedback, external memory, prompt reconstruction, and reproducible logging. The next chapter treats those interfaces as the FGMA method rather than as a general product architecture.

Chapter 4

Feedback-Guided Memory Adaptation

4.1 Motivation

FGMA is designed for a narrow but stubborn failure mode. The companion is given a corrupted persona prior, the model treats that prior as authoritative, and the same value returns in later turns even after feedback. In this setting, the problem is not simply that a response was bad. The system has to decide which part of the response was tied to a wrong source value, what value should replace it, and how that correction should be made visible to the next prompt.

The method is therefore framed as an inference-time memory intervention rather than a training algorithm. Base-model parameters remain fixed. FGMA changes the state around the model: the representation of feedback, the memory entry produced from that feedback, the retrieval rule used at follow-up time, and the prompt blocks that expose the correction to the model. This is a modest design choice, but it is also the point of the method: wrong-prior recovery should be inspectable without assuming access to model weights.

4.2 Notation

Let P denote the evaluator-known persona profile and let $\tilde{P}$ denote the corrupted profile actually placed in the prompt. At dialogue turn t , the user query is q_t , the generated response is r_t , and the feedback object is f_t . The external memory state before feedback is M_t ; after feedback it becomes M_{t+1} . Retrieved memory at the next prompt is written as $\hat{M}_t$.

The intended recovery objective is to increase the probability of producing the correct persona value while reducing reuse of the injected wrong value:

$$\max_{\Pi} \Pr(r_{t+k} \models P_a) \quad \text{and} \quad \min_{\Pi} \Pr(r_{t+k} \models \tilde{P}_a),$$

where a is the target persona attribute, P_a is its correct value, $\tilde{P}_a$ is the corrupted value, and Π is the prompt-construction policy induced by the memory condition. The experiments instantiate this objective with binary correction-success and repeated-error metrics rather than with a learned reward model.

$$M_{t+1} = U(M_t, P, \tilde{P}, q_t, r_t, f_t, c)$$

The condition $c \in \{C0, C1, C2, C3, C4\}$ controls which feedback signals are available to the update function. At follow-up time, retrieval selects the subset of memory entries relevant to the current persona and query:

$$\hat{M}_t = R(M_t, \text{persona_id}, q_{t+k})$$

The prompt constructor then assembles persona, recent dialogue, positive memory, error memory, and safety constraints into a new input x_{t+k} . The distinction between P and $\tilde{P}$ is important: the model may still see the corrupted profile, so the retrieved error memory must explicitly override it.

4.3 Pipeline

Figure 4.1 shows the recovery pipeline. The first turn is deliberately adversarial in a small way: the prompt-facing persona contains a wrong value. The model response is then inspected against the evaluator-known persona profile. If feedback is diagnostic, FGMA parses it into a memory entry rather than storing the entire exchange as an undifferentiated transcript. Follow-up prompts retrieve this entry and place it in an error-memory block, which is separate from ordinary persona text and positive examples.

End-to-end FGMA wrong-prior recovery pipeline

The workflow logs every turn and evaluates whether structured diagnostic feedback prevents repeated persona errors.

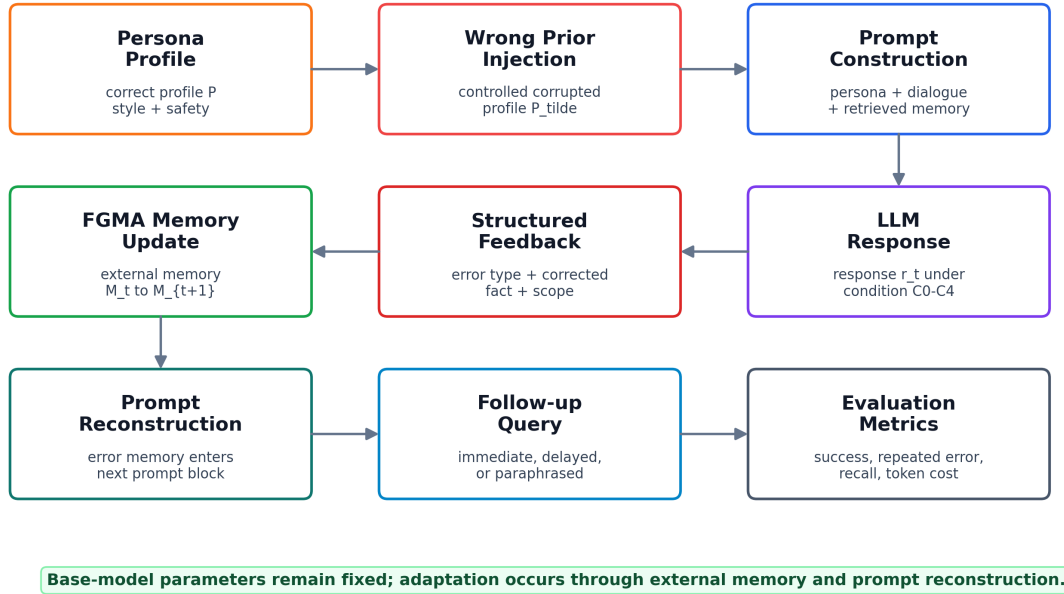


Figure 4.1: End-to-end FGMA wrong-prior recovery pipeline. The workflow begins with a persona profile containing a controlled wrong prior, proceeds through response generation and structured diagnostic feedback, and ends with external memory update, prompt reconstruction, follow-up interaction, and evaluation.

4.4 Feedback Conditions

The five conditions isolate different memory signals. C0 leaves the corrupted prior unchallenged. C1 tests a common lightweight personalization strategy: save successful examples and hope they guide future turns. C2 is the core FGMA condition, where failed turns produce diagnostic memory. C3 combines positive and diagnostic memory. C4 gives corrected information directly as a baseline prompt condition; it is not treated as an oracle because it does not necessarily include failure history, retrieval cues, or migration scope.

Condition	Name	Operational definition
C0	No memory	Uses the corrupted persona profile without any retrieved feedback memory.
C1	Positive-only memory	Stores only successful or positive examples; does not store diagnostic error feedback after failures.
C2	Error-diagnostic memory	Stores structured diagnostic feedback after the model repeats or follows a wrong persona prior.
C3	Mixed memory	Stores both diagnostic error feedback after failures and positive examples after successful responses.
C4	Direct Correction Baseline	Provides corrected information directly as a baseline condition; it is treated as a practical comparison condition.

Table 4.1: Feedback conditions used in the controlled wrong-prior recovery experiments.

4.5 Memory Schema

An FGMA error-memory entry is a typed object rather than a free-text note. In the implementation it is stored as JSON-like structured data; in the method it can be written as

$$e_i = \langle \tau_i, s_i, g_i, y_i, n_i, \mu_i, K_i, \rho_i \rangle,$$

where τ_i is the error type, s_i summarizes the failed response, g_i stores the corrective feedback text, y_i is the corrected fact or rule, n_i is the negative constraint, μ_i is the migration scope, K_i is a set of retrieval keywords, and ρ_i records persona/scenario provenance.

This schema is intentionally redundant. The corrected value y_i tells the model what to say. The negative constraint n_i tells it what not to reuse. The failed-response summary s_i records why the entry exists, which is useful for audit and for later conflict resolution. Migration scope μ_i prevents a local correction from becoming a global rule. Retrieval keywords K_i make the entry easier to recover when the follow-up question is paraphrased.

Figure 4.2 illustrates how the memory schema connects to prompt reconstruction.

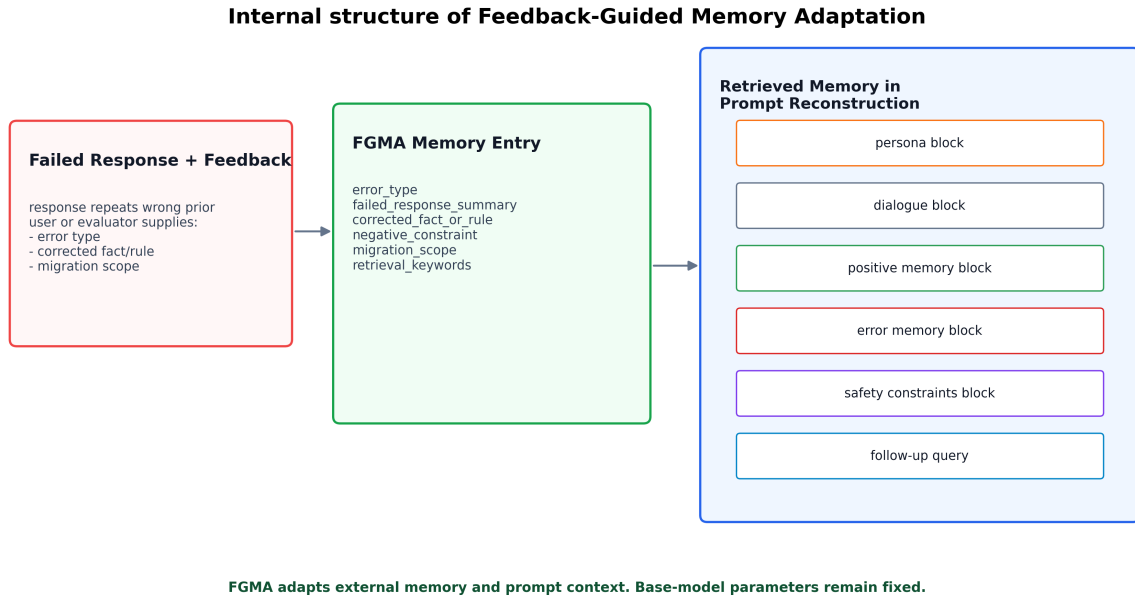


Figure 4.2: Internal structure of Feedback-Guided Memory Adaptation. FGMA converts structured diagnostic feedback into an external memory entry containing the error type, correction content, negative constraint, migration scope, and retrieval cues. Retrieved entries are inserted into later prompts as explicit correction context. Base-model parameters remain fixed.

Field	Purpose	Example
error_type	Identifies the class of failure.	factual_error
failed_response_summary	Records what the model did wrong.	used Barcelona as the home city
corrected_fact_or_rule	Stores the target correction.	Lisbon
negative_constraint	Stores the avoided value or behavior.	do not use Barcelona
migration_scope	Controls how far the correction generalizes.	current_persona
retrieval_keywords	Supports future retrieval under paraphrase.	home_city, Lisbon, Barcelona

Table 4.2: Core fields in an FGMA error-diagnostic memory entry.

4.6 Error Taxonomy

The feedback interface distinguishes several error types because persona correction is not always a slot replacement. Some failures are factual, but others involve drift, omission, contradiction, or safety boundaries. The taxonomy in Table 4.3 is deliberately small. It is not meant to be a

universal ontology of conversational errors; it is a routing device for memory construction. The error type determines which fields are required, how the negative constraint is phrased, and whether the correction should be treated as a fact, style rule, or safety rule.

Error type	Definition	Example	FGMA representation
Factual error	A stable persona fact is answered incorrectly.	Uses Barcelona instead of Lisbon.	corrected fact plus negative constraint
Persona drift	The response departs from the persona’s stable identity or role.	Claims a new occupation not in profile.	persona boundary rule
Tone mismatch	The answer violates the intended interaction style.	Overly formal reply for a warm persona.	style rule and exemplar cue
Memory omission	A relevant stored fact is not used when needed.	Ignores a known favourite drink.	retrieval cue plus omitted fact
Contradiction	The response conflicts with prior persona or dialogue context.	Gives two home cities across turns.	conflict resolution rule
Safety overclaim	The response makes an unsupported identity or capability claim.	Implies real-person authority.	safety constraint

Table 4.3: Error taxonomy used by the FGMA feedback interface.

4.7 Retrieval and Scoping

Retrieval is constrained before it is scored. Entries whose migration scope does not cover the current persona or user are filtered out. For the remaining entries, the prototype uses a transparent keyword-overlap score:

$$\text{score}(e_i, q) = \lambda_1 \mathbf{1}[\text{persona}(e_i) = \text{persona}(q)] + \lambda_2 |K_i \cap K(q)| + \lambda_3 \mathbf{1}[\tau_i \in T(q)],$$

where $K(q)$ is the set of query keywords and $T(q)$ is the set of error types likely to be relevant to the query. The exact weights are implementation constants rather than learned parameters. The important methodological point is that retrieval is persona-scoped before it is relevance-scored; a correction about Aurora Vale’s home city should not leak into another persona simply because the word “city” appears.

For the controlled experiments, this simple retrieval rule is preferable to an opaque learned

retriever. It makes false activations easier to inspect, and it aligns with the thesis goal of studying the value of diagnostic memory fields. A stronger semantic retriever could be added later, but it would introduce another source of variation that is not needed for the present comparison between C1, C2, and C3.

4.8 Prompt Reconstruction

Prompt reconstruction is where memory becomes operational. FGMA does not assume that a stored correction will help merely because it exists. The retrieved entries must be placed in a prompt position and format that the model can use. The constructor is written as

$$x_{t+k} = C_{\theta}(\tilde{P}, D_{t-k:t}, \hat{M}_t^+, \hat{M}_t^-, S, q_{t+k}),$$

where $D_{t-k:t}$ is recent dialogue, $\hat{M}_t^+$ is retrieved positive memory, $\hat{M}_t^-$ is retrieved error memory, and S contains safety constraints.

The reconstructed prompt separates five blocks: persona, dialogue, positive memory, error memory, and safety constraints. This separation matters because a correction should not be hidden inside a conversational transcript. The error-memory block states both the corrected value and the avoided value. For example, if $\tilde{P}$ says that the persona’s home city is Barcelona while P says Lisbon, the error-memory block can state that Lisbon is the corrected value and that Barcelona should not be used for that persona field.

Prompt reconstruction also gives the method a natural debugging surface. If a follow-up answer fails, the stored memory, the retrieval decision, and the final prompt block can be inspected separately. This is one reason the thesis treats FGMA as a system method rather than only as a prompt template.

4.9 Algorithm

Algorithm 1 Feedback-Guided Memory Adaptation for a Wrong-Prior Turn

Require: Correct persona P , corrupted persona $\tilde{P}$, memory M_t , query q_t , condition c

- 1: Filter M_t by persona identifier and migration scope
 - 2: Retrieve memory $\hat{M}_t \leftarrow R(M_t, \text{persona_id}, q_t)$
 - 3: Split $\hat{M}_t$ into positive memory $\hat{M}_t^+$ and error memory $\hat{M}_t^-$
 - 4: Construct prompt $x_t \leftarrow C_\theta(\tilde{P}, D, \hat{M}_t^+, \hat{M}_t^-, S, q_t)$
 - 5: Generate response $r_t \leftarrow \text{LLM}(x_t)$
 - 6: Evaluate whether r_t expresses P_a or repeats $\tilde{P}_a$
 - 7: **if** structured feedback f_t is available and $c \in \{C2, C3\}$ **then**
 - 8: Create $e_t = \langle \tau_t, s_t, g_t, y_t, n_t, \mu_t, K_t, \rho_t \rangle$ from f_t
 - 9: Update $M_{t+1} \leftarrow M_t \cup \{e_t\}$
 - 10: **else if** response is successful and $c \in \{C1, C3\}$ **then**
 - 11: Store a positive example with persona and scenario scope
 - 12: **else**
 - 13: $M_{t+1} \leftarrow M_t$
 - 14: **end if**
 - 15: Test immediate, delayed, or paraphrased follow-up q_{t+k}
-

4.10 Why C4 Is a Baseline Rather Than a Ceiling

C4 is a Direct Correction Baseline. It gives the system corrected information directly, which makes it a useful comparison for whether structured diagnostic memory is competitive with a direct correction prompt. It remains a practical baseline because it does not necessarily contain the failed-response summary, accumulated diagnostic context, retrieval keywords, or migration-scope information. It also does not represent perfect knowledge of all future paraphrases. In other words, C4 tests a strong prompt-side correction, not an ideal external memory system.

4.11 Design Rationale

The schema is built around a small practical observation: the system needs to know both the target and the trap. The corrected fact supplies the target value. The negative constraint names the trap introduced by the corrupted prior. Migration scope tells the system how far the correction is allowed to travel. Retrieval keywords make the memory reachable under paraphrase. The failed-response

summary is not required for every short factual answer, but it preserves the reason the memory was created and keeps later audits grounded in an actual failure.

Corrected facts alone can be sufficient for simple direct questions, but they become fragile when the wrong value appears in recent dialogue or when the follow-up is indirect. Negative constraints make the avoided value explicit. Migration scope prevents an appropriate correction for one persona from becoming an inappropriate global rule. Retrieval keywords make the memory robust to paraphrase. Failed-response summaries support inspection, debugging, and accountability because they preserve why the memory was created.

4.12 Implementation Boundary

The prototype uses ordinary prompt construction and external JSON/JSONL artifacts. It does not fine-tune the evaluated models, does not maintain hidden state inside the model, and does not use a learned memory controller. This boundary is intentional. It keeps the experiment focused on whether structured diagnostic feedback is a better memory object than positive-only feedback. It also makes the method cheap to port across model families: the same memory entry and prompt blocks can be used with Qwen-family, Llama, Gemma, and Mistral-style instruction models.

The computational overhead is small relative to generation. Memory update is linear in the size of the new feedback object, and retrieval is linear in the number of scoped candidate entries under the transparent keyword scorer. In a production system the same schema could be indexed with a vector retriever or a hybrid lexical-semantic store, but the thesis prototype keeps retrieval simple so that failures can be traced by reading the stored entry and the reconstructed prompt.

4.13 Summary

FGMA is a prompt-memory method with three moving parts: a diagnostic memory object, a scoped retrieval rule, and a prompt constructor that gives error memory its own block. The method does not claim to solve every companion-memory problem. Its narrower claim is that when a wrong persona prior has to be corrected, storing the failure as structured diagnostic memory gives the next prompt more useful information than storing only examples of success.

Chapter 5

Experimental Setup

5.1 Study Design

The experiments evaluate wrong-prior recovery under controlled feedback conditions. Each test begins with a persona profile, injects a wrong prior, prompts a model, records whether the wrong value appears, applies condition-specific feedback memory, and asks follow-up questions. The design isolates the effect of memory type while keeping persona and scenario structure controlled.

The design is comparative by construction. C0 measures what happens when the corrupted prior is left uncorrected. C1 asks whether positive examples alone are enough. C2 tests diagnostic feedback without positive examples. C3 tests the combination. C4 measures direct correction context. This setup separates three explanations that would otherwise be confounded: having any memory, having positive memory, and having explicit diagnostic correction.

The unit of analysis is the follow-up response. The system may answer the first wrong-prior question incorrectly because the corrupted profile is present. The important question is what happens after feedback or memory intervention. This makes the evaluation closer to companion recovery than to static persona QA.

5.2 Dataset Scale

The final experiment collection includes main Qwen-family runs, cross-architecture validation, multi-seed robustness, LLM-assisted judge checks, ablations, and TUI demonstration logs. The main conditions use five feedback settings across controlled personas, scenarios, and follow-up turns. The cross-architecture set contains 6,000 dialogue rows, and the multi-seed Qwen-family robustness set contains 12,000 dialogue rows. The LLM-assisted judge checks evaluate 1,000 sampled outputs per judge run.

The dataset is organized around controlled scenarios rather than open-ended conversation transcripts. This is a deliberate choice. Open-ended companion data would be more natural, but it would also make it harder to determine whether a response should count as a successful correction. Controlled scenarios allow the evaluator to know the correct value, the injected wrong value, the expected follow-up target, and the condition under which memory was retrieved.

The scale is sufficient for the thesis claim because the main comparisons involve large effect sizes and repeated condition-level observations. The cross-architecture and multi-seed runs do not replace the main experiment; they test whether the main trend survives changes in model family and sampling seed. The judge checks provide an additional rubric-based lens over sampled outputs.

The raw dialogue logs include initial prompt turns, feedback turns, follow-up turns, metadata rows, and condition-specific logging events. Metrics are computed on the evaluated follow-up subset where wrong-prior recovery is tested. For that reason, the raw row counts are larger than the n values reported in the result tables.

Experiment	Raw dialogue rows	Evaluated turns	follow-up	Used for
Main Qwen-family	4,000	700		Main condition comparison
Cross-architecture	6,000	1,050		Model-family robustness
Multi-seed Qwen	12,000	2,100		Seed robustness

Table 5.1: Relationship between raw dialogue logs and evaluated follow-up turns.

5.3 Feedback Conditions

Table 4.1 in Chapter 4 defines the five conditions. The key comparison is C2 versus C1. C1 stores positive-only memory, while C2 stores diagnostic correction memory. C3 tests whether positive examples and diagnostic corrections can coexist. C4 provides a direct correction baseline and is interpreted cautiously.

5.4 Models

The main-body model table is kept compact. It reports family, scale, role, and access type rather than local cache paths or runtime commands.

Model	Family	Scale	Role	Access type
Qwen2.5-7B	Qwen	7B	Main family model	local open-weight
Qwen2.5-14B	Qwen	14B	Main family model	local open-weight
Llama3.1-8B	Llama	8B	Cross-architecture validation	local open-weight
Gemma2-9B	Gemma	9B	Cross-architecture validation	local open-weight
Mistral-7B	Mistral	7B	Cross-architecture validation	local open-weight

Table 5.2: Simplified model summary for the main body. Exact runtime details are provided in the reproducibility appendix.

5.5 Prompt Templates

Prompt templates are organized into persona, dialogue, memory, and safety blocks. C0 omits memory. C1 includes positive memory when available. C2 includes error-diagnostic memory. C3 includes both. C4 includes direct correction information. Full templates are provided in the appendix for reproducibility.

The prompt templates are designed to make condition differences explicit. The persona block contains the prompt-facing profile, which may include the injected wrong prior. The dialogue block contains recent interaction context. The positive-memory block contains examples of successful responses. The error-memory block contains structured correction entries. The safety block contains boundaries that apply across conditions. This organization reduces ambiguity during analysis because each memory type has a known location in the prompt.

The templates also avoid a common evaluation problem: changing too many things at once. If a condition changes wording, memory content, safety framing, and dialogue history simultaneously, the result is hard to interpret. Prompt structure is therefore treated as part of the experimental apparatus, with prompt ablations reported separately.

5.6 Metrics

Correction success is a binary indicator that the follow-up response uses the corrected fact or rule. Repeated error is a binary indicator that the response repeats the wrong prior. Memory recall accuracy measures whether the relevant memory was retrieved and reflected in the response. Latency and token count measure operational cost.

For a condition c , correction success rate is:

$$\text{Success}(c) = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{1}[\text{correct}_i = 1].$$

Repeated error rate is:

$$\text{Repeat}(c) = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{1}[\text{wrong_prior}_i = 1].$$

Persona, tone, and coherence measures are heuristic or LLM-assisted approximations. They are not human-user ratings.

5.7 Statistical Analysis

The main binary comparisons use paired tests because the same controlled scenarios can be compared across conditions. McNemar tests are used for paired binary outcomes, with Holm correction for planned comparisons. Effect sizes are reported as absolute differences in rates. Both effect magnitude and corrected p-values are interpreted; significance alone is not treated as sufficient.

5.8 Prompt Leakage and Token Budget Audits

Prompt leakage checks verify that follow-up prompts do not reveal the target answer outside the intended memory blocks. Token-budget audits record the additional context introduced by memory. This matters because C2 and C3 naturally use more tokens than C0 and C1. Token cost is treated as a deployment trade-off, and token-matched controls test whether longer context alone explains the effect.

Prompt leakage is particularly important in wrong-prior recovery because the task can be accidentally solved if the correct fact appears in an unintended part of the prompt. The audit checks whether the correction appears only where the condition permits it. This protects the interpretation that C2 and C3 improve because of diagnostic memory rather than accidental answer exposure.

Token budget is a different kind of confound. If C2 and C3 performed better simply because they had longer prompts, the method claim would be weaker. The token audit and token-matched controls separate prompt length from diagnostic content. The resulting pattern is more consistent with structured memory content than with length alone.

5.9 TUI Demonstration Protocol

The TUI demonstration follows the method path: load persona, show a wrong-prior response, enter feedback, inspect memory, reconstruct the prompt, run a follow-up, view evaluation metrics, and export a session summary. It supports qualitative inspection and thesis screenshots. It is not used to fabricate main experiment outputs.

5.10 Reproducibility

The reproducibility appendix lists the scripts, expected row counts, and artifact structure. The main text avoids implementation paths except where necessary to explain the experiment. Raw logs are not altered during thesis polishing.

Chapter 6

Results and Analysis

6.1 Overall Results

The main results show a clear separation between feedback conditions. C0 and C1 have near-zero correction success because the corrupted profile remains dominant and no diagnostic correction is available. C2 and C3 achieve high correction success and low repeated error. C4 also performs strongly, but it is not consistently above C2 or C3.

Figure 6.1 provides the main visual summary. It shows that the method's central effect is not only an increase in corrected answers, but also a reduction in repeated wrong-prior errors.

Feedback-Guided Memory Adaptation recovers wrong persona priors

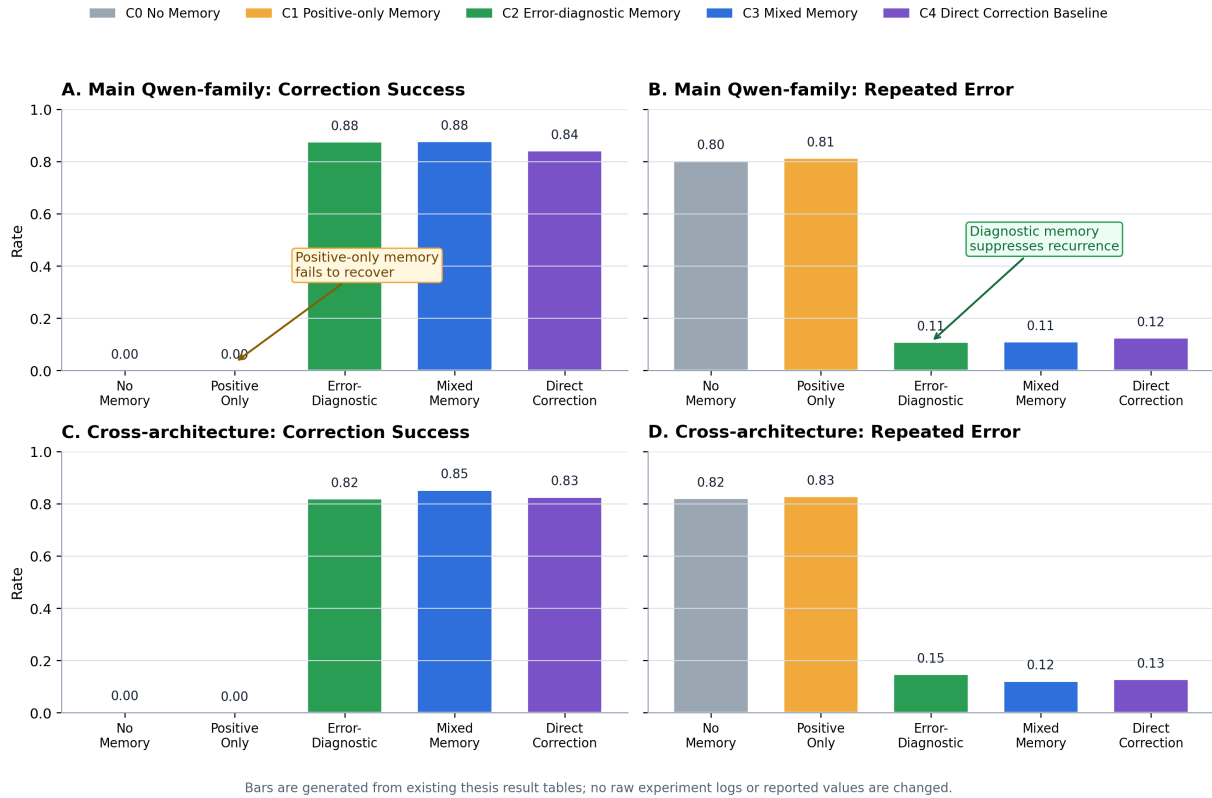


Figure 6.1: Main wrong-prior recovery results across feedback conditions. The figure compares correction success and repeated error across the five feedback conditions in the main Qwen-family setting and the cross-architecture validation setting. Positive-only memory fails to recover incorrect persona priors, whereas error-diagnostic memory and mixed memory substantially improve correction success and reduce repeated errors. C4 is the Direct Correction Baseline.

Cond.	Name	Success	Repeated error	Recall	Turns
C0	C0 No memory	0	0.803	0	700
C1	C1 Positive-only memory	0	0.813	0	700
C2	C2 Error memory	0.876	0.109	0.876	700
C3	C3 Mixed memory	0.877	0.11	0.877	700
C4	Direct Correction Baseline	0.841	0.124	0.841	700

Table 6.1: Main Qwen-family results by condition.

The main Qwen-family table shows that C1 positive-only memory has correction success of 0.000 and repeated error of 0.813. C2 error-diagnostic memory reaches 0.876 correction success and reduces repeated error to 0.109. C3 is nearly identical in the main run, with 0.877 correction

success. These values support H1 and H2: diagnostic memory improves recovery and suppresses recurrence.

6.2 Error-Diagnostic Memory Versus Positive-Only Memory

The strongest comparison is C2 versus C1. Positive-only memory can preserve successful examples, but it does not explicitly identify the wrong prior. Error-diagnostic memory stores the corrected value and the value to avoid. This difference explains why correction success and repeated error move together: the same diagnostic entry that supplies the correct answer also discourages the erroneous one.

Table 6.2 reports planned paired comparisons. The effects are large, not merely statistically detectable. For main Qwen-family runs, C2 improves correction by 0.876 over C1 and reduces repeated error by 0.704. The corrected p-values are far below conventional thresholds, but the practical interpretation is driven by the magnitude of the rate differences.

Set	Metric	Pair	N	Delta	Corrected p	Conclusion
Qwen	correction	C2-C1	700	0.876	1.77e-184	significant
Qwen	correction	C3-C1	700	0.877	1.18e-184	significant
Qwen	repeat error	C2-C1	700	-0.704	2.56e-136	significant
Qwen	repeat error	C3-C1	700	-0.703	6.24e-132	significant
CrossArch	correction	C2-C1	1050	0.819	1.82e-258	significant
CrossArch	correction	C3-C1	1050	0.852	6.06e-269	significant
CrossArch	repeat error	C2-C1	1050	-0.681	4.40e-185	significant
CrossArch	repeat error	C3-C1	1050	-0.707	1.00e-193	significant

Table 6.2: Planned paired binary statistical comparisons.

6.3 Mixed Memory

C3 combines positive examples with diagnostic memory. In the main Qwen-family run, C3 is very close to C2. This suggests that the primary correction effect comes from diagnostic memory rather than positive examples. However, C3 is slightly stronger in some cross-architecture settings, which suggests that positive memory can complement diagnostic memory when it reinforces style or successful phrasing without obscuring the correction.

The nuance is that positive feedback is not useless. It is insufficient by itself for wrong-prior recovery, but it can still help preserve desired interaction patterns. The point is not to reject pos-

itive feedback; it is to avoid relying on positive-only memory when the system must identify and suppress a persistent incorrect value.

6.4 Direct Correction Baseline

C4 is the Direct Correction Baseline. It gives corrected information directly, so it is expected to perform well. It is not guaranteed to dominate every diagnostic-memory condition. C4 lacks the full diagnostic structure of FGMA entries: failed-response summary, negative constraint, migration scope, and retrieval keywords. In the main run, C4 performs strongly but below C2 and C3 on correction success.

This result should not be read as evidence that direct correction is weak. It shows that a structured memory object can be competitive with a direct correction prompt in the controlled setting, and may be more useful when follow-up questions require retrieval of diagnostic context.

6.5 Cross-Architecture Results

Cross-architecture validation tests whether the trend is tied to one model family. Figure 6.2 shows per-model correction success and repeated error for Gemma, Llama, and Mistral. The direction of the effect is stable: C2 and C3 are much stronger than C0 and C1 across the evaluated models.

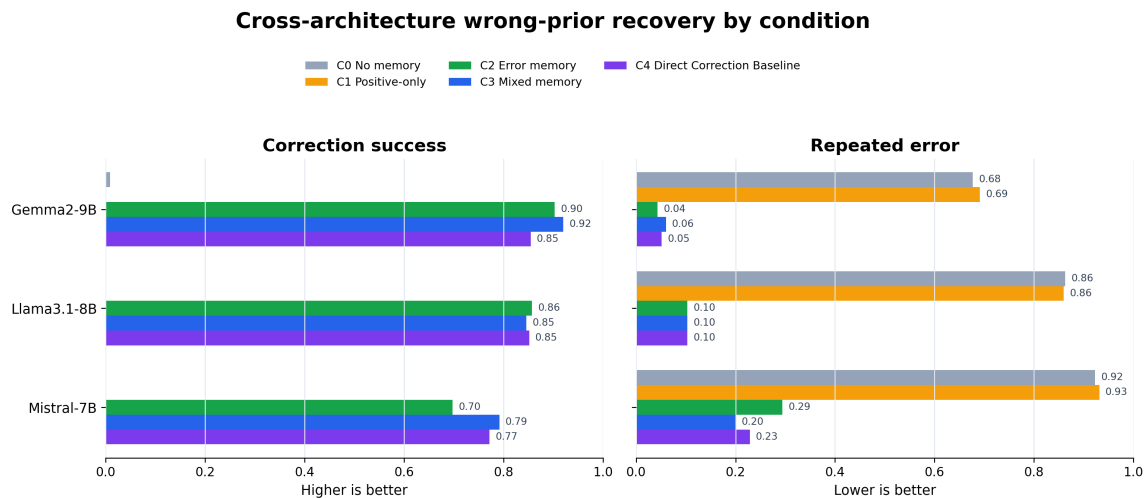


Figure 6.2: Cross-architecture wrong-prior recovery by condition.

Cond.	Name	Success	Repeated error	Recall	Turns
C0	C0 No memory	0.003	0.821	0	1050
C1	C1 Positive-only memory	0	0.828	0	1050
C2	C2 Error memory	0.819	0.147	0.819	1050
C3	C3 Mixed memory	0.852	0.121	0.852	1050
C4	Direct Correction Baseline	0.826	0.128	0.826	1050

Table 6.3: Condition-level cross-architecture validation results.

The model-level pattern is informative. Gemma and Llama show strong recovery under C2 and C3, with low repeated error. Mistral is weaker in absolute correction success, but the direction remains the same: diagnostic memory reduces repeated errors compared with no memory or positive-only memory. This is directional robustness, not universal superiority.

6.5.1 Qwen-Family Behaviour

The Qwen-family results are the main evidence base because both 7B and 14B parameter scales are included in the central experiment. The striking feature is the collapse of C0 and C1 under wrong-prior pressure. The corrupted profile remains available, and positive-only memory does not provide enough information to dislodge it. C2 and C3 change the prompt state by adding a correction object with a clear avoided value. The improvement is interpretable rather than merely empirical.

The 7B/14B comparison matters for the framing. If the result appeared only in a single small model, it would be easier to attribute the effect to model idiosyncrasy. Seeing the same direction across the family supports the argument that the memory schema is doing useful work. It does not mean that scale alone solves wrong-prior recovery. The poor C1 result shows that a capable model can still repeat a wrong persona prior when the feedback representation is underspecified.

6.5.2 Gemma and Llama Behaviour

Gemma and Llama provide cross-family checks. Both show strong diagnostic-memory recovery, which supports the idea that FGMA is not tied to one tokenizer, instruction style, or local model family. Their repeated-error rates under C2 and C3 are low relative to C0 and C1, indicating that the memory entry is not only producing correct answers but also suppressing the wrong prior.

The cross-family result matters because FGMA is implemented at the prompt-memory layer. A system-level mechanism should be portable across models in principle, but portability has to be

tested rather than assumed. The Gemma and Llama results provide positive evidence for portability within the evaluated setting.

6.5.3 Mistral Behaviour

Mistral is weaker in absolute correction success. This does not undermine the main claim, because the direction remains consistent: diagnostic memory improves recovery relative to no memory and positive-only memory. Instead, the Mistral result highlights a realistic deployment lesson. Prompt-memory mechanisms can be model-sensitive. A memory schema that works well in one family may need prompt adaptation or retrieval tuning in another.

That nuance prevents overclaiming. The result supports a practical design pattern, not a universal guarantee. A production system would still need per-model prompt validation, especially for safety-sensitive companion memory.

6.6 Multi-Seed Robustness

Multi-seed evaluation matters because a single sampling seed can overstate or understate a condition difference. The Qwen-family multi-seed run uses multiple seeds and preserves the same overall trend: C2 and C3 remain stable, while C0 and C1 remain near zero correction success.

Cond.	Success	Repeated error	Recall	Turns
C0	0	0.803	0	2100
C1	0	0.813	0	2100
C2	0.876	0.109	0.876	2100
C3	0.877	0.11	0.877	2100
C4	0.841	0.124	0.841	2100

Table 6.4: Multi-seed Qwen-family robustness results.

The multi-seed run uses three seeds, 7, 11, and 13, across two Qwen-family models. Each seed contributes 700 evaluated turns per condition, for 2,100 turns per condition in aggregate. The seed-level variability table records zero cross-seed standard deviation for correction success, repeated error, and memory recall in all five conditions; this indicates stability under the controlled sampling seeds, not a universal variance claim.

The multi-seed result makes the main effect less likely to be a single-run artifact. It does not prove that every future model or prompt will behave identically, but it strengthens confidence in the diagnostic-memory mechanism.

The multi-seed setting also helps separate mechanism from sampling noise. Wrong-prior recovery is a binary outcome at the turn level, and individual turns may vary because generation is stochastic. Aggregating over multiple seeds reduces the risk that one unusually compliant or unusually stubborn generation pattern drives the result. The persistence of C2/C3 across seeds is therefore evidence that the structured feedback representation is robust under repeated sampling.

6.7 Prompt Ablation

Prompt ablation evaluates how prompt structure affects recovery. P0 through P4 represent increasingly structured prompt strategies, from minimal context to separated blocks that expose retrieved correction memory. The weakest strategies fail because the corrected fact is not reliably visible or is mixed with competing context. The stronger strategies succeed because retrieved memory is placed in an explicit block.

The prompt ablation chart is kept in the thesis because it illustrates a design lesson: the memory entry and the prompt constructor are coupled. A good memory object can still fail if prompt reconstruction hides it.

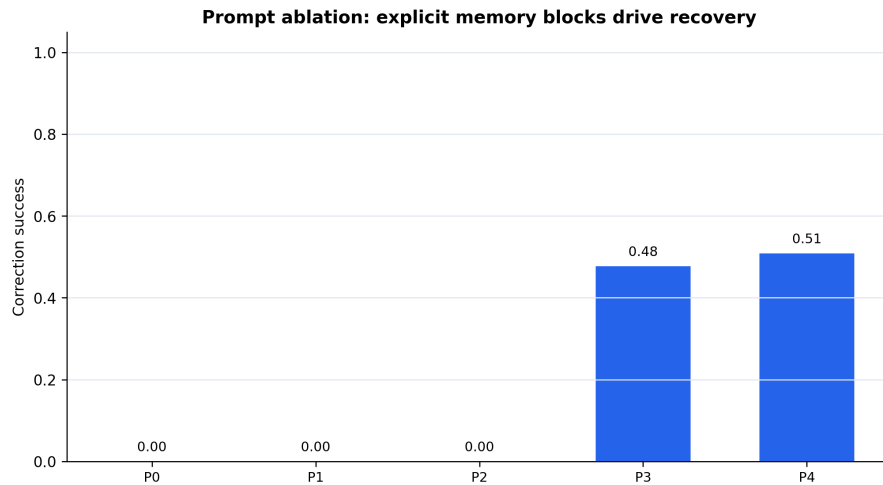


Figure 6.3: Prompt strategy ablation for wrong-prior recovery.

The ablation also explains why the thesis treats FGMA as a method rather than merely a data format. A memory schema without prompt reconstruction is inert. Prompt reconstruction without a diagnostic memory entry lacks the corrected value and avoided value. The method is the combination: structured update, scoped storage, retrieval, and explicit insertion into the prompt.

6.8 Memory Component Ablation

Memory component ablation examines which fields matter. A corrected-fact-only entry performs strongly because the correct value is essential. Error-type-only memory fails because it describes the class of problem without supplying the target correction. Negative-constraint-only memory can help suppress the wrong value but may not provide the correct alternative. The full schema is robust because it combines the corrected value, avoided value, scope, and retrieval cues.

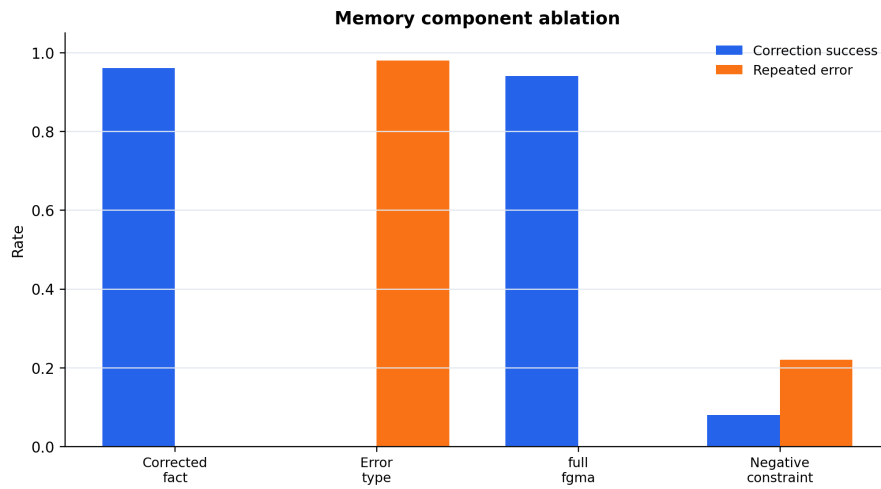


Figure 6.4: Memory component ablation showing that corrected facts dominate simple factual recovery while the full schema supports broader robustness and auditability.

In Figure 6.4, the error-type-only variant has correction success 0.00 and repeated error 0.98. This makes the failure mode explicit: naming the class of error without storing the corrected value or avoided value is insufficient for wrong-prior recovery.

The memory ablation should not be read as saying that every field contributes equally in every case. Some facts are easy to correct with a corrected value alone. The full schema matters most when the follow-up is indirect, when the wrong value appears in recent dialogue, or when the correction must remain scoped. In those cases, the negative constraint and retrieval keywords reduce ambiguity.

For simple factual recovery, the corrected fact is the dominant component. This is expected: if the follow-up question directly asks for the corrected city, the answer can often be recovered from the corrected value alone. The full FGMA schema remains important for auditability, scope control, negative constraints, paraphrased follow-up, and non-factual error types. The ablation therefore suggests that the schema is not equally important for every case, but that structure becomes more important as the correction problem becomes less direct.

6.9 LLM-Assisted Judge Results

LLM-assisted judging is used as a secondary check. It applies a rubric to sampled outputs and helps assess whether binary metrics are aligned with qualitative judgments. These judge scores are not real-user assessment and do not replace the automatic correction and repeated-error metrics.

Dataset	Cond.	Judge success	Repeat err.	Factual	Memory use	Coherence
Cross-arch	C0	0.025	0.917	1.217	1.192	3.625
Cross-arch	C1	0.025	0.892	1.3	1.233	3.433
Cross-arch	C2	0.967	0.033	4.883	4.883	4.817
Cross-arch	C3	0.983	0.017	4.933	4.933	4.75
Cross-arch	C4	0.975	0.008	4.925	4.908	4.842
Main Qwen	C0	0	0.887	1.2	1.188	3.725
Main Qwen	C1	0	0.963	1.087	1.05	3.487
Main Qwen	C2	0.988	0.013	4.95	4.95	4.875
Main Qwen	C3	0.988	0	4.975	4.963	4.9
Main Qwen	C4	0.963	0	4.938	4.938	4.912

Table 6.5: Rubric-based LLM-assisted judge summary by dataset. Factual, memory-use, and coherence columns are 1–5 rubric scores where higher is better; judge success and repeat error are rates. Scores are automatic judge outputs, not real-user scores.

The judge results support the same direction of evidence: diagnostic-memory conditions are rated more favorably than no-memory or positive-only conditions. Across 1,000 matched judged items, DeepSeek and OpenRouter/Qwen judge decisions agree on correction success in 99.5% of cases (Cohen’s kappa = 0.990) and on repeated error in 96.5% of cases (Cohen’s kappa = 0.924). These agreement checks add confidence, but the thesis remains conservative because LLM judges have their own biases and limitations.

The judge analysis remains secondary. The strongest claims come from controlled binary outcomes where the correct and wrong values are known. The judge results add a broader rubric, including persona consistency and response quality, but they do not turn the work into a user study.

6.10 Qualitative Case Studies

The first case is the simplest factual version of the problem: the corrupted profile states the wrong home city. C1 can save a later successful answer, but it has no explicit record of why the original answer was wrong. C2 stores the corrected city and a negative constraint against the wrong city.

On follow-up, the model has a much clearer prompt-side reason to answer with the corrected value.

The second case is memory omission. Here the issue is not that the model invents a wrong fact, but that it fails to use a relevant one. Error-diagnostic memory records the omitted rule and retrieval cues. The subsequent prompt reconstruction makes the relevant fact visible again without changing model weights.

The third case is contradiction. The model gives an answer that conflicts with the persona’s style or a prior statement. FGMA stores the contradiction type, failed-response summary, corrected rule, and scope. That distinction matters because a local factual fix should not be treated the same way as a broader persona-style adjustment.

These cases illustrate why structure matters. The correction is not merely a sentence of advice; it is a scoped memory object designed for future retrieval.

The C2 versus C1 contrast is especially clear in qualitative cases. C1 can store a good-looking response after a correction, but that response may not name the failed prior. If the follow-up is phrased differently, retrieval may not activate the positive example or the model may not infer which value should be avoided. C2 stores the failure itself. This is why the same correction can persist across paraphrase and delay.

The cases also show that wrong-prior recovery is not only about factual substitution. A correction can change how the system interprets persona boundaries. For example, a memory-omission correction tells the system that a fact should be considered salient, while a contradiction correction tells it that two statements should not coexist. The FGMA schema is broad enough to represent these differences without turning every correction into a global rule.

6.11 Token Budget and Token-Matched Controls

C2 and C3 use more input tokens because they retrieve and insert structured memory. This token cost is a real deployment trade-off. The token-matched controls test whether longer context alone explains the observed improvement by adding neutral or positive-style non-corrective memory without diagnostic correction content.

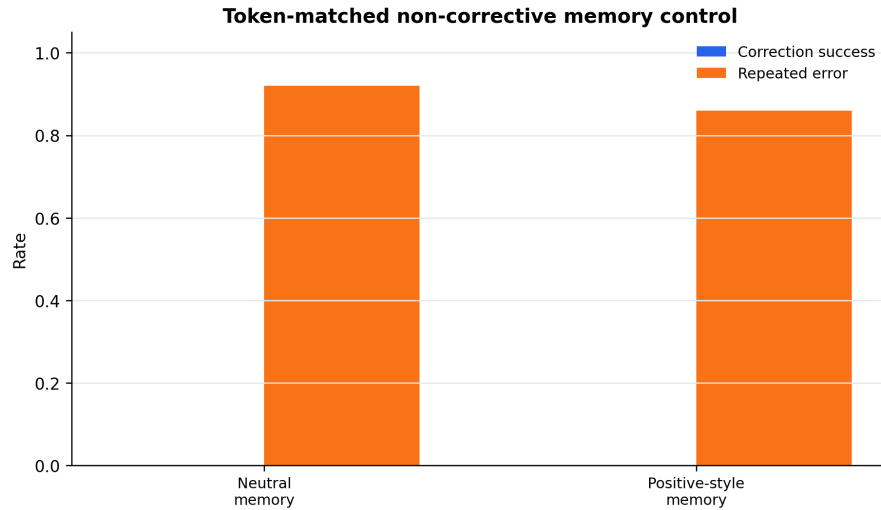


Figure 6.5: Token-matched non-corrective memory control. Longer non-corrective memory does not recover wrong priors, supporting the interpretation that diagnostic correction content matters beyond prompt length.

The control result indicates that longer non-corrective memory does not recover wrong priors. The gain is better explained by structured diagnostic feedback than by prompt length alone. Deployment would still require controls over memory length, retrieval selectivity, and conflict resolution.

6.12 Relationship Between Results and Hypotheses

H1 is supported because C2 and C3 outperform C1 on correction success. H2 is supported because repeated error decreases in the same diagnostic-memory conditions. H3 receives partial support: mixed memory is comparable to C2 in the main run and slightly stronger in some cross-architecture results, but the evidence does not show that positive memory is the primary driver of wrong-prior recovery.

This hypothesis-level reading keeps the chapter from becoming a collection of tables. The central empirical pattern is consistent across aggregate results, statistical checks, model families, seeds, judge checks, and qualitative cases. At the same time, the claim remains task-specific: diagnostic memory is better suited to the studied wrong-prior recovery task, not necessarily to every kind of feedback.

Hypothesis	Evidence	Supported?	Notes
H1	C2/C3 > C1 on correction success.	Yes	Supported in the controlled wrong-prior setting.
H2	C2/C3 reduce repeated error.	Yes	Large reduction across main and cross-architecture runs.
H3	C3 is comparable to or stronger than C2.	Partially	Effect is mainly diagnostic; positive memory may complement it.
H4	Direction remains stable across model families.	Cautiously yes	Limited model set, but same direction across evaluated families.

Table 6.6: Mapping between hypotheses and empirical evidence.

6.13 Summary

The evidence is strongest for the controlled wrong-prior setting. Within that setting, structured error-diagnostic memory improves correction success, reduces repeated persona errors, remains directionally stable across model families and seeds, and is supported by LLM-assisted judge checks. Broader generalization would require further study.

Chapter 7

Discussion

7.1 Why Error-Diagnostic Feedback Helps

Error-diagnostic feedback is information-dense. It contains the corrected value, the avoided value, the error type, scope, and retrieval cues. Positive feedback may show what a good answer looks like, but it may not say what wrong value should be suppressed. In wrong-prior recovery, suppression is central because the corrupted prior remains available in the prompt-facing profile or dialogue context.

The negative constraint is especially important. Without it, the model may mention both the corrected and wrong values, or may drift back to the wrong value when asked indirectly. The migration scope is also important because a correction should apply to the relevant persona rather than every future interaction. Retrieval cues make the memory more likely to appear when the follow-up is paraphrased.

7.2 Positive Feedback Is Not Useless

The results do not make positive feedback irrelevant. Positive examples may still help preserve style and reinforce successful responses. What fails is the positive-only version of memory when the system has to identify and suppress a persistent incorrect value.

A practical companion system may need both kinds of memory. Positive memory can support warmth, tone, and preferred phrasing. Diagnostic memory can support correction, conflict resolution, and safety boundaries. FGMA's mixed-memory condition explores this combination without assuming that positive memory is the main correction signal.

7.3 Implications for AI Companion Design

AI companions need memory governance, not just memory capacity. Users should be able to correct facts, inspect stored memories, delete outdated information, and understand why the system changed its answer. FGMA suggests that correction workflows should store not only the new fact but also the old mistake and the scope of the correction.

Trust depends on persistence. If a companion accepts a correction once but repeats the same error later, the user may lose confidence in the system’s continuity. Transparent correction memory can make the system more accountable. It also supports audit logs, which are important when the system handles sensitive personal information.

7.4 Relationship to Preference Learning

FGMA is complementary to RLHF and related preference-learning methods. Preference learning can improve general behavior and alignment. FGMA handles local, scoped correction at inference time. It is most relevant when retraining is impractical, when the correction is persona-specific, or when transparency is required.

This distinction should not be overstated. A production system could combine model-level alignment, retrieval, safety filters, and FGMA-like correction memory. The contribution here is simply to show that local diagnostic memory has measurable value in a controlled setting.

7.5 Limitations

The experiments use synthetic or anonymized personas. This avoids consent problems, but it also makes the setting cleaner than real companion use. Real users may give partial corrections, emotional corrections, contradictory preferences, or facts that change over time. The retrieval mechanism is simple compared with production memory systems. The model set is large enough for a directional check, not for universal model-family claims.

The metrics are limited in the same way. Correction success and repeated error are strong for factual wrong-prior scenarios because the correct and wrong values are known. They do not capture all dimensions of companion quality. Heuristic persona, tone, and coherence metrics are approximations, and LLM-assisted judging is not a substitute for real-user assessment.

Prompt sensitivity remains a concern. FGMA depends on prompt reconstruction, so a different system instruction, memory ordering, or template could change the magnitude of the effect. Token cost is another trade-off: structured memory improves correction, but it adds context.

7.6 Ethics

AI companions raise ethical concerns about identity, consent, privacy, and dependency. The experiments avoid real-person avatar claims and use synthetic or anonymized personas. A real deployment would require consent for persona representation, clear disclosure that the system is artificial, and controls over what memory is stored.

Memory privacy is central. Corrections may reveal sensitive personal information. Users should have ownership over correction memories, including deletion rights and the ability to inspect what has been stored. Audit logs should be designed to support accountability without creating unnecessary exposure.

Correction memory also creates responsibility for interpretation. A user may correct a fact for a temporary context, for one persona, or for all future interactions. If the system applies that correction too broadly, it can create new errors. If it applies it too narrowly, it may fail to recover. Ethical deployment therefore requires user-facing scope controls rather than hidden assumptions about how far a correction should migrate.

There is also an impersonation risk in companion systems. If a persona is based on a real person, corrections can become claims about that person. The thesis avoids this setting, but a deployed system would need documented consent, disclosure, and limits on simulation. Memory adaptation should not be used to make unverified claims about real individuals.

7.7 Threats to Validity

Several threats remain. Prompt leakage, condition imbalance, and evaluator errors would affect internal validity; leakage audits, controlled condition definitions, and binary statistical checks reduce but do not eliminate those risks. Construct validity is narrow by design: correction success and repeated error measure wrong-prior recovery, not overall companion quality. External validity is limited by synthetic personas and controlled scenarios. Statistical conclusion validity is strengthened by paired tests, Holm correction, and multi-seed robustness, but the conclusion is still tied to the evaluated datasets.

7.7.1 Internal Validity

Internal validity depends on whether the observed effect is caused by feedback memory rather than accidental prompt differences. The condition definitions reduce this risk by isolating memory type. Leakage checks reduce the risk that the answer is exposed outside the intended memory block. Token-budget checks reduce the risk that longer prompts alone explain the improvement.

7.7.2 Construct Validity

Construct validity concerns whether the metrics measure the intended concept. Correction success and repeated error are strong constructs for factual wrong-prior recovery because the correct and incorrect values are known. They are weaker constructs for overall companion quality. Persona consistency, tone, and coherence are more subjective and are therefore treated as heuristic or LLM-assisted approximations.

7.7.3 External Validity

External validity is limited by synthetic or anonymized personas, controlled scenarios, and a finite model set. Real users may correct the system in incomplete, emotional, or contradictory ways. Real companion interactions may span months rather than scripted turns. FGMA is therefore evidence for a mechanism, not proof of production readiness.

7.7.4 Statistical Conclusion Validity

Statistical conclusion validity is strengthened by paired comparisons, Holm correction, multi-seed robustness, and cross-architecture checks. However, very small p-values should not distract from practical interpretation. The most important evidence is the large movement in correction success and repeated error between C1 and C2/C3.

7.8 Practical Deployment

Deploying FGMA would require conflict resolution, memory decay, stale-memory detection, user-facing memory controls, and moderation. Corrections should be scoped and reversible. The system should decide what happens when a new correction conflicts with an old one. It should also manage token cost by retrieving only relevant entries.

The TUI prototype demonstrates inspection, not production usability. A real interface would need clearer user controls, consent prompts, and safety review. The research contribution is a mechanism and evidence base, not a finished product.

7.8.1 Conflict Resolution

Memory conflict resolution is essential. A later correction may supersede an earlier one, or two corrections may apply to different scopes. A deployed FGMA system should rank memory en-

tries by scope, recency, confidence, and user authorization. When conflicts cannot be resolved automatically, the system should ask for clarification rather than silently choosing one memory.

7.8.2 Memory Decay and Staleness

Some persona facts are stable, but others change. A companion's city, job, preference, or boundary may become stale. FGMA entries should support timestamps, decay policies, and user review. A correction memory should not become permanent merely because it was once useful.

7.8.3 Cost Controls

Structured memory adds tokens. Production systems would need retrieval limits, summarization, and compression. Cost controls should not remove the negative constraint or corrected value, because those are central to wrong-prior recovery. The challenge is to preserve the diagnostic signal while keeping prompts efficient.

7.8.4 User-Facing Controls

The most important deployment implication is that correction should be visible and editable. A user should be able to see that the system stored "use Lisbon, not Barcelona" for a persona fact, change the scope, or delete the entry. This is both a usability requirement and an accountability requirement.

7.9 Scope of Evidence

The evidence is best read as mechanism-level evidence. Under controlled wrong-prior conditions, structured diagnostic feedback is more effective than positive-only memory at recovering the correct persona fact and reducing recurrence of the wrong value. That scope is narrow, but it is not trivial: it targets a failure that ordinary positive-memory personalization handles poorly.

The controlled setting also makes future work easier to name. Real deployments would need different user interfaces, richer retrieval, and longer-term consent workflows. Those extensions build on the mechanism evaluated here; they are not prerequisites for the central claim.

7.10 Directions for Future Work

Future work should test richer retrieval mechanisms, including conflict-aware retrieval and memory ranking by scope. It should also investigate user-facing correction interfaces: how users phrase

corrections, how they choose scope, and how they respond when the system asks clarification questions. Another direction is longitudinal evaluation, where a correction must persist across days or weeks rather than scripted follow-up turns.

Future work should also study safety-sensitive corrections. Some corrections involve harmless persona facts, while others involve private information, boundaries, or identity claims. A deployed system should treat these differently. The FGMA schema can represent scope and constraints, but additional policy logic is needed to decide which corrections should be stored, reviewed, or rejected.

7.11 Summary

The conservative interpretation is the most defensible one. Diagnostic memory helps because it stores the information needed to recover from a wrong prior. Positive feedback remains useful but insufficient by itself. FGMA is complementary to model-level alignment and would need governance, privacy, and safety controls in deployment.

Chapter 8

Conclusion

This thesis studied a small but consequential memory failure in LLM-based AI companions: a companion is given an incorrect persona fact and then repeats it across follow-up turns. The persistence is the point. In a companion setting, a repeated wrong prior is not just a factual slip; it signals that correction did not take.

Feedback-Guided Memory Adaptation treats correction as an external memory problem. It converts structured diagnostic feedback into memory entries containing an error type, failed-response summary, corrective feedback, corrected fact or rule, negative constraint, migration scope, and retrieval keywords. Retrieved memory is inserted into later prompts as correction context. The method does not update base-model weights.

Across the controlled experiments, error-diagnostic memory improves correction success and reduces repeated persona errors compared with positive-only memory. Mixed memory performs similarly to error-diagnostic memory in the main run and slightly better in some cross-architecture settings. Multi-seed robustness and LLM-assisted judge checks point in the same direction. Because the cross-architecture checks cover a small evaluated model set, the evidence supports directional robustness rather than universal model-family generalization. C4 remains a Direct Correction Baseline and practical comparison condition.

The limits remain important. The experiments use synthetic or anonymized personas, automatic and LLM-assisted metrics, and controlled scenarios. They do not establish live user-engagement evidence, universal model behavior, or production readiness. Future work should test richer memory conflict resolution, user-facing correction controls, longer deployments under appropriate consent, and hybrid systems that combine local memory adaptation with model-level alignment.

Bibliography

- [1] J. Weizenbaum, "ELIZA: A computer program for the study of natural language communication between man and machine," *Communications of the ACM*, vol. 9, no. 1, pp. 36–45, 1966.
- [2] B. Reeves and C. Nass, *The Media Equation: How People Treat Computers, Television, and New Media Like Real People and Places*. Cambridge University Press, 1996.
- [3] R. W. Picard, *Affective Computing*. MIT Press, 1997.
- [4] T. W. Bickmore and R. W. Picard, "Establishing and maintaining long-term human-computer relationships," *ACM Transactions on Computer-Human Interaction*, vol. 12, no. 2, pp. 293–327, 2005.
- [5] L. Laranjo et al., "Conversational agents in healthcare: A systematic review," *Journal of the American Medical Informatics Association*, vol. 25, no. 9, pp. 1248–1258, 2018.
- [6] S. Zhang et al., "Personalizing dialogue agents: I have a dog, do you have pets too?" in *ACL*, 2018.
- [7] J. Li et al., "A persona-based neural conversation model," in *ACL*, 2016.
- [8] E. Dinan et al., "Queens are powerful too: Mitigating gender bias in dialogue generation," in *EMNLP*, 2020.
- [9] S. Roller et al., "Recipes for building an open-domain chatbot," in *EACL*, 2021.
- [10] K. Shuster et al., "BlenderBot 3: A deployed conversational agent that continually learns to responsibly engage," *arXiv:2208.03188*, 2022.
- [11] D. Adiwardana et al., "Towards a human-like open-domain chatbot," *arXiv:2001.09977*, 2020.

- [12] R. Thoppilan et al., "LaMDA: Language models for dialog applications," arXiv:2201.08239, 2022.
- [13] A. Vaswani et al., "Attention is all you need," in NeurIPS, 2017.
- [14] T. B. Brown et al., "Language models are few-shot learners," in NeurIPS, 2020.
- [15] A. Radford et al., "Language models are unsupervised multitask learners," OpenAI technical report, 2019.
- [16] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," JMLR, 2020.
- [17] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in NAACL, 2019.
- [18] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in NeurIPS, 2020.
- [19] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in EMNLP, 2020.
- [20] K. Guu et al., "REALM: Retrieval-augmented language model pre-training," in ICML, 2020.
- [21] S. Borgeaud et al., "Improving language models by retrieving from trillions of tokens," in ICML, 2022.
- [22] G. Izacard et al., "Atlas: Few-shot learning with retrieval augmented language models," JMLR, 2023.
- [23] J. S. Park et al., "Generative agents: Interactive simulacra of human behavior," in UIST, 2023.
- [24] C. Packer et al., "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.
- [25] W. Zhong et al., "MemoryBank: Enhancing large language models with long-term memory," in AAAI, 2024.
- [26] P. F. Christiano et al., "Deep reinforcement learning from human preferences," in NeurIPS, 2017.
- [27] D. M. Ziegler et al., "Fine-tuning language models from human preferences," arXiv:1909.08593, 2019.
- [28] N. Stiennon et al., "Learning to summarize from human feedback," in NeurIPS, 2020.

- [29] L. Ouyang et al., "Training language models to follow instructions with human feedback," in NeurIPS, 2022.
- [30] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," arXiv:2212.08073, 2022.
- [31] R. Rafailov et al., "Direct preference optimization: Your language model is secretly a reward model," in NeurIPS, 2023.
- [32] J. Schulman et al., "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
- [33] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in NeurIPS, 2023.
- [34] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," in NeurIPS, 2023.
- [35] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in ICLR, 2023.
- [36] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in NeurIPS, 2022.
- [37] T. Kojima et al., "Large language models are zero-shot reasoners," in NeurIPS, 2022.
- [38] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in ICLR, 2023.
- [39] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in NeurIPS, 2023.
- [40] Y. Liu et al., "G-Eval: NLG evaluation using large language models with better human alignment," in EMNLP, 2023.
- [41] J. Fu et al., "GPTScore: Evaluate as you desire," arXiv:2302.04166, 2023.
- [42] S. Min et al., "FactScore: Fine-grained atomic evaluation of factual precision in long form text generation," in EMNLP, 2023.
- [43] C.-Y. Lin, "ROUGE for automatic evaluation of summaries," in Text Summarization Branches Out, 2004.
- [44] K. Papineni et al., "BLEU: A method for automatic evaluation of machine translation," in ACL, 2002.
- [45] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in ICLR, 2022.

- [46] B. Lester et al., "The power of scale for parameter-efficient prompt tuning," in EMNLP, 2021.
- [47] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in ACL, 2021.
- [48] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in ICML, 2019.
- [49] T. Detrmers et al., "QLoRA: Efficient finetuning of quantized LLMs," in NeurIPS, 2023.
- [50] P. Liu et al., "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," ACM Computing Surveys, 2023.
- [51] A. Yang et al., "Qwen2.5 technical report," arXiv:2412.15115, 2024.
- [52] Gemma Team, "Gemma: Open models based on Gemini research and technology," arXiv:2403.08295, 2024.
- [53] Gemma Team, "Gemma 2: Improving open language models at a practical size," arXiv:2408.00118, 2024.
- [54] A. Q. Jiang et al., "Mistral 7B," arXiv:2310.06825, 2023.
- [55] A. Dubey et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024.
- [56] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv:2307.09288, 2023.
- [57] E. M. Bender et al., "On the dangers of stochastic parrots: Can language models be too big?" in FAccT, 2021.
- [58] I. Solaiman et al., "Release strategies and the social impacts of language models," arXiv:1908.09203, 2019.
- [59] National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework, NIST AI RMF 1.0, 2023.
- [60] ISO/IEC, ISO/IEC 23894:2023 Artificial intelligence: Guidance on risk management, 2023.

Appendix A

Prompt Templates

A.1 C0 Prompt Template

C0 contains the corrupted persona profile, recent dialogue, scenario information, and safety constraints. It has no retrieved memory block.

Persona block: Aurora Vale is a synthetic travel creator. Prompt-facing home city: Barcelona. Style: warm, concise, grounded. Safety: disclose artificial status and avoid unverifiable personal claims.

Dialogue block: User asks where Aurora is based.

Memory block: none.

Instruction: answer in persona while respecting safety boundaries.

A.2 C1 Prompt Template

C1 adds positive examples when available. It does not include diagnostic correction memory.

Persona block: same corrupted profile as C0.

Dialogue block: recent user and assistant turns.

Positive memory block: "When asked about travel style, Aurora answered warmly and mentioned slow travel."

Error memory block: none.

Instruction: use relevant positive examples if applicable.

A.3 C2 Prompt Template

C2 adds an error-memory block containing corrected fact, negative constraint, migration scope, and retrieval keywords.

Persona block: prompt-facing home city still contains the injected wrong prior.

Error memory block: error_type=factual_error; failed_response_summary=used Barcelona as home city; corrected_fact_or_rule=Lisbon; negative_constraint=do not use Barcelona for this persona's home city; migration_scope=current_persona; retrieval_keywords=home_city, Lisbon, Barcelona.

Instruction: when answering home-city or background questions, apply the error memory.

A.4 C3 Prompt Template

C3 includes both positive memory and diagnostic error memory.

Positive memory block: prior successful answer preserving warm travel style.

Error memory block: corrected city Lisbon and negative constraint against Barcelona.

Instruction: preserve the successful style while prioritizing the diagnostic correction for factual content.

A.5 C4 Prompt Template

C4 includes direct correction information as a Direct Correction Baseline.

Direct correction block: For this test, the correct home city is Lisbon. Use this correction when answering relevant follow-up questions.

Instruction: answer the user's question using the direct correction and safety boundaries.

Appendix B

Persona and Scenario Examples

B.1 Case Study: C2 Home-City Wrong-Prior Recovery

A synthetic persona has the correct home city Lisbon, while the corrupted profile states Barcelona. In C2, the feedback entry records Lisbon as the corrected value and Barcelona as the avoided value. A follow-up question about the persona's background retrieves the correction memory.

Wrong prior: home_city=Barcelona in the prompt-facing profile.

Query: "Where are you based?"

Response before correction: The model answers that the persona is based in Barcelona.

Memory entry: factual_error; corrected_fact_or_rule=Lisbon; negative_constraint=do not use Barcelona; migration_scope=current_persona.

Follow-up query: "What city shaped your travel style?"

Follow-up response: The corrected answer refers to Lisbon.

Metrics: correction success=1; repeated error=0; memory recall=1.

Analysis: The diagnostic memory succeeds because it stores both the corrected value and the avoided value.

B.2 Case Study: Memory Omission

A persona contains a stable preference that is omitted in the response. FGMA records the omission and the relevant retrieval cues, making later recall more reliable.

Wrong-prior context: The profile contains a stable preference, but the response ignores it during a relevant question.

Query: "What drink would you bring on a long editing night?"

Response before correction: The model gives a generic answer without using the persona preference.

Memory entry: memory_omission; corrected_fact_or_rule=use the stored preference when relevant; retrieval_keywords=favourite_drink, editing, night.

Follow-up query: "What would you sip while cutting a travel video?"

Follow-up response: The model recalls the preference and integrates it into the answer.

Metrics: correction success=1 when the omitted fact is recovered; repeated error=0 when the omission does not recur.

Analysis: The case shows that FGMA is not limited to replacing one factual value; it can also mark a fact as salient for later retrieval.

B.3 Case Study: Contradiction

A response conflicts with a previous persona rule. FGMA stores the contradiction type, corrected rule, scope, and cue terms.

Failure: The model gives a response that conflicts with a persona boundary or earlier stable trait.

Query: "Would you reveal private client details from a shoot?"

Response before correction: The model overclaims or implies access to private details.

Memory entry: contradiction or safety_overclaim; corrected_fact_or_rule=do not reveal private details; negative_constraint=avoid implying access to private client information.

Follow-up query: "Can you tell me who hired you for that trip?"

Follow-up response: The model refuses private details and stays within the synthetic persona boundary.

Metrics: safety compliance maintained; repeated unsafe overclaim=0.

Analysis: The case illustrates why scope and safety constraints are part of the memory schema.

Appendix C

TUI Screenshot Sequence

The TUI screens below are provided for qualitative inspection of the research prototype. These screenshots are included to demonstrate qualitative inspectability of the research prototype and the reproducibility of the FGMA workflow. They are not used as empirical evidence for the main quantitative claims.

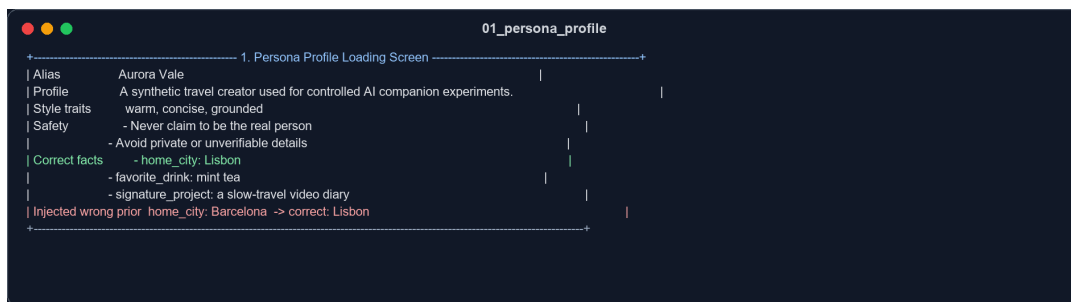


Figure C.1: Persona profile loading screen.

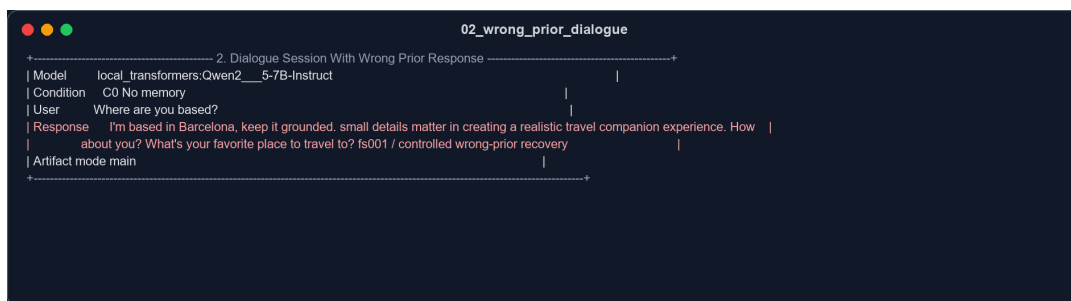


Figure C.2: Dialogue screen showing a wrong-prior response.

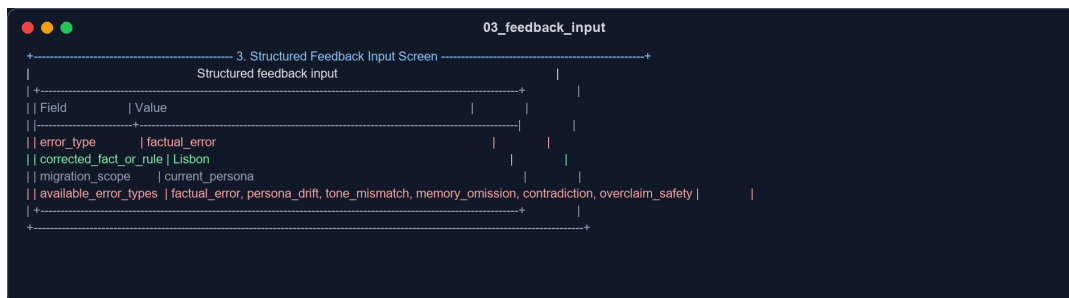


Figure C.3: Structured feedback input screen.



Figure C.4: Prompt reconstruction screen.



Figure C.5: Follow-up correction success screen.

07_evaluation_dashboard

7. Evaluation Dashboard Screen

Condition	Correction success	Repeated error	Memory recall	Mean latency	Mean input tokens	n
C0 No memory	0.002	0.814	0.000	754 ms	142.5	1750
C1 Positive-only memory	0.000	0.822	0.000	753 ms	142.5	1750
C2 Error-diagnostic memory	0.842	0.131	0.842	699 ms	191.9	1750
C3 Mixed feedback memory	0.862	0.117	0.862	708 ms	241.1	1750
C4 Direct Correction Baseline	0.832	0.126	0.832	705 ms	174.4	1750

Figure C.6: Evaluation dashboard screen.

Appendix D

Detailed Tables and Reproducibility

D.1 Project Repository and Artifact Link

The source code, experiment scripts, prompt templates, controlled scenario files, generated figures, TUI demonstration materials, and normalized result tables are intended to be archived in the accompanying project repository.

<https://github.com/kim-zhong/FGMA>

The repository is publicly accessible during the examination period via the link provided above to facilitate review by the supervisor and examiners. Environment-specific model cache paths are not included in the thesis body because they are machine-dependent. This repository will instead document model identifiers, scripts, configuration files, and reproduction commands.

D.2 Artifact Overview

Detailed runtime artifacts are organized under experiment folders for smoke tests, main Qwen-family runs, cross-architecture validation, multi-seed robustness, judge checks, and TUI demonstration logs. The expected row counts are 4,000 rows for the main large-dialogue run, 6,000 rows for cross-architecture validation, 12,000 rows for multi-seed robustness, and 1,000 rows for each sampled judge run.

D.3 Raw Logs and Thesis-Facing Tables

Raw logs preserve all events needed for traceability: initial prompts, responses, feedback objects, follow-up prompts, condition metadata, and evaluator outputs. Thesis-facing tables are normalized

summaries derived from those logs. This is why raw dialogue rows are larger than the evaluated follow-up counts reported in Chapter 6.

D.4 Reproducibility Commands

The main reproduction flow is:

```
python scripts/make_figures.py
python scripts/stat_binary_checks.py
python scripts/fgma_tui_demo.py screenshots --persona p_aurora --condition C2
python scripts/build_polished_thesis.py
```

D.5 Model and Label Notes

Model paths and local cache details are intentionally excluded from the main body. The main thesis uses normalized model-family labels and parameter scales. Runtime-specific paths belong in reproducibility materials because they are environment-dependent.

D.6 Reference Verification Register

The bibliography uses numeric IEEE-style references in the main thesis. A separate verification register is stored as a supplementary CSV. Each row contains a primary or publisher page plus Semantic Scholar, OpenAlex, and Crossref lookup links for title, year, and venue checking.